



*Asociación Profesional de Cuerpos Superiores
de Sistemas y Tecnologías de la Información
de las Administraciones Públicas.*

Temario para la preparación de la Oposición al Cuerpo Superior de
Sistemas y Tecnologías de la Información de la Administración del
Estado.

TEMAS ESPECÍFICOS – BLOQUE B

III INGENIERÍA DE LOS SISTEMAS DE INFORMACIÓN

**Tema 98. Métricas y evaluación de la calidad del software. La
implantación de la función de calidad**

AUTOR: Pedro-Jesús Pastor Muñoz

REVISOR: Elisa Martín Ortega

Creación: septiembre 2023

**Actualización: septiembre
2003**

ÍNDICE

1.	INTRODUCCIÓN Y CONCEPTOS.....	4
2.	EL SW COMO PRODUCTO DE CALIDAD.....	6
3.	ESTÁNDARES NACIONALES E INTERNACIONALES.....	8
4.	MÉTRICAS Y EVALUACIÓN DE LA CALIDAD DEL SOFTWARE.....	10
4.1	DEFINICIONES PREVIAS	10
	ACOPLAMIENTO (<i>COUPLING</i>).....	10
	COHESIÓN (<i>COHESION</i>)	10
	COMPLEJIDAD (<i>COMPLEXITY</i>)	11
	CAMBIOS DE CÓDIGOS (<i>CODE CHURN</i>).....	11
	CÓDIGO CUBIERTO (<i>CODE COVERAGE</i>)	11
	CÓDIGO MUERTO (<i>DEAD CODE</i>),.....	12
	CÓDIGO DUPLICADO (<i>CODE DUPLICATION</i>).....	12
	CÓDIGO DUDOSO (<i>DODGY CODE, BAD CODE</i>)	13
	TÉRMINOS SOBRE RENDIMIENTO DE ALGORITMOS	13
	TÉRMINOS SOBRE CORRECCIÓN DE ERRORES.....	13
4.2	MÉTRICAS SOBRE EL PRODUCTO ANALIZADO.....	13
4.2.1	SOBRE DOCUMENTACIÓN ASOCIADA	13
4.2.2	DE CÓDIGO FUENTE.....	14
4.2.3	DE DISEÑO.....	14
4.2.4	DE ARQUITECTURA.....	15
4.3	MÉTRICAS DE ASPECTOS ESPECÍFICOS.....	16
4.4	OTRAS MÉTRICAS DEL PROCESO DE DESARROLLO	18
5.	IMPLANTACIÓN DE LA FUNCION DE CALIDAD	19
5.1	PRINCIPIOS DE CALIDAD	19
5.1.1	PRINCIPIOS SOLID	19
	PRINCIPIO DE RESPONSABILIDAD ÚNICA	19
	PRINCIPIO DE ABIERTO/CERRADO: <i>OPEN/CLOSED PRINCIPLE (OCP)</i>	19
	PRINCIPIO DE SUSTITUCIÓN DE LISKOV.....	19
	PRINCIPIO DE SEGREGACIÓN DE LA INTERFAZ:.....	20
	PRINCIPIO DE INVERSIÓN DE DEPENDENCIAS.	20
5.1.2	PATRONES DE DISEÑO	20
	PATRONES DE CREACIÓN	20
	PATRONES DE COMPORTAMIENTO.....	21
5.2	ERRORES DEL SOFTWARE MÁS COMUNES.....	22

5.3	PRUEBAS DE SW	22
6.	LA IMPLANTACIÓN DE LA FUNCIÓN DE CALIDAD.....	24
6.1	CONSTITUCIÓN DE ESTÁNDARES, NORMAS, ETC.....	24
6.2	CONSTITUCIÓN DEL EQUIPO DE GESTIÓN DE CALIDAD	25
6.3	IMPLANTACIÓN DE CALIDAD POR PROYECTO SEGÚN METRICA	26
6.3.1	ESTUDIO DE VIABILIDAD DEL SISTEMA	26
6.3.2	ANÁLISIS DEL SISTEMA DE INFORMACIÓN.....	27
6.3.3	DISEÑO DEL SISTEMA DE INFORMACIÓN.....	28
6.3.4	CONSTRUCCIÓN DEL SISTEMA DE INFORMACIÓN	29
6.3.5	IMPLANTACIÓN Y ACEPTACIÓN DEL SISTEMA.....	30
6.3.6	MANTENIMIENTO DEL SISTEMA DE INFORMACIÓN	31
7.	HERRAMIENTAS DE CALIDAD DEL SW	33
8.	CONCLUSIONES	36
9.	RESUMEN ESQUEMÁTICO.....	37
10.	GLOSARIO.....	38
11.	TEST	39
11.1	PREGUNTAS DE TEST.....	39
11.2	SOLUCIONES A LAS PREGUNTAS DE TEST	40
12.	BIBLIOGRAFÍA BÁSICA.....	41
12.1	BIBLIOGRAFÍA BÁSICA	41
12.2	BIBLIOGRAFÍA PARA AMPLIAR EL TEMA.....	41

1. INTRODUCCIÓN Y CONCEPTOS

Primero definiremos qué se entiende por calidad, en general: “grado en que un conjunto de características inherentes cumple con unos requisitos” [ISO 9000:2000], en nuestro caso, que se cumplen requisitos de nuestros usuarios para el funcionamiento de dichos sistemas de información. Por su parte, el “Aseguramiento de la Calidad” consiste en los métodos usados en el proceso de desarrollo de software que controlan que el sistema desarrollado verifica dichos requisitos de calidad.

La necesidad de desarrollar software, SW, de calidad surge como respuesta a la “Crisis del SW” de los primeros años del desarrollo de la informática. Cada intento de desarrollar sistemas más potentes y complejos resultaba un incremento exponencial del esfuerzo, tanto del coste como del tiempo de desarrollo. Esto llevó a la implementación de modelos de desarrollo en los que se definieron fases diferenciadas, tareas específicas en cada una de ellas y productos intermedios a los que controlar su calidad. Igualmente se definieron métricas de calidad del código para mejorar el producto final.

Además, los sistemas de información perduran durante mucho tiempo y están sometidos a cambios de legislación, de criterio de gestión, y a nuevas necesidades de sus usuarios. Los datos perdurarán incluso más que el propio sistema de gestión. Por ello, resulta especialmente interesante que no solo se verifiquen los requisitos funcionales del sistema, sino también la facilidad para su posterior corrección, adaptación y evolución.

La calidad se basa en estándares predefinidos. No se puede valorar un cumplimiento de normas de nomenclatura en un código fuente si no se han definido previamente. No se puede calificar una adecuada estructuración o separación del código en módulos o clases si no se definen previamente los criterios más adecuados. Algunos lenguajes de programación ayudan en la definición de dichos estándares y facilitan esta estructuración. Además, resulta imprescindible contar con normas propias de cada organización.

Por otro lado, las metodologías de desarrollo de SW, como Métrica V3, definen la calidad como un conjunto de tareas que se ejecutan de manera paralela a la gestión del proyecto y que nos ayudan a definir un plan específico de aseguramiento de calidad aplicables a dicho proyecto. “En el plan de aseguramiento de calidad se reflejan las actividades de calidad a realizar (normales o extraordinarias), los estándares a aplicar, documentos a revisar, los procedimientos a seguir en la obtención de los distintos productos durante el desarrollo en MÉTRICA Versión 3 y la normativa para informar de los defectos detectados a sus responsables y realizar el seguimiento de los mismos hasta su corrección “.

Entre las actividades normales: “el plan de aseguramiento de calidad comienza en el Estudio de Viabilidad del Sistema y se aplica a lo largo de todo el desarrollo, en los procesos de Análisis, Diseño, Construcción, Implantación y Aceptación del Sistema y en su posterior Mantenimiento.” Propone reuniones intermedias para asegurar que todas las partes del proyecto validan los productos intermedios o en sus primeras etapas de desarrollo. También recomiendan la existencia de un plan de aseguramiento de calidad previo para cualquier proyecto y nos ayudan en su elaboración.

Igualmente proponen procedimientos extraordinarios, como las auditorías, aplicables en desarrollos singulares y en el transcurso de las cuales se revisarán tanto las actividades de desarrollo como las propias de aseguramiento de calidad.

Como se ha indicado, el proceso de construcción de un sistema de gestión resulta complejo y se aborda por fases intermedias que generan subproductos parciales: primero las fases de análisis, después diseño, etc., generando cada una un conjunto de documentos y programas usados en las siguientes fases. Por ello, el aseguramiento de la calidad se debe dar en las diferentes fases de la metodología usada, especialmente en el código fuente, manuales, instrucciones de instalación, pero también debemos analizar la calidad los subproductos intermedios obtenidos y la relación y coherencia entre ellos.

Además, detectar un error en las fases previas resulta mucho más barato que propagar estos errores o desvíos a fases posteriores del desarrollo, ya que las correcciones serán más complejas, supondrán cambios en más elementos y requerirán más tiempo.

Otro elemento que ayuda unívocamente al control de calidad es el control del presupuesto. Inicialmente los controles de calidad suponen aumentar los elementos de control sobre los productos entregados. Y esto incrementa el tiempo de desarrollo y el coste del producto. No obstante, la implantación de calidad se trata de un cambio a gestionar en cada equipo de desarrollo y en todo el ciclo de vida. Finalmente, la calidad mejora el valor del producto SW final, por lo que compensa los costes iniciales de su implantación.

Tras esta pequeña introducción estamos en situación de definir la calidad de manera genérica como: "Conjunto de propiedades y características de un producto o servicio, que le confieren aptitud para satisfacer una serie de necesidades explícitas o implícitas" (ISO 8402)". En el caso específico de calidad del software se puede definir también como: "Conformidad a los requisitos funcionales y prestaciones establecidas, a las normas de desarrollo explícitamente documentadas y a las características implícitas que se esperan de todo software desarrollado profesionalmente" (Roger R. Pressman). A lo que hay que añadir que esta calidad se debe asegurar en condiciones de facilitar el mantenimiento y evolución posterior del sistema de información.

Se pueden destacar varios apartados de esta definición:

- La calidad se define inicialmente como la adecuada funcionalidad de los sistemas de información.
- También se requiere calidad en productos intermedios (para su posterior mantenimiento) y en otros productos finales como manuales, guías, etc. (para su implantación).
- Se requieren normas de desarrollo previas para facilitar la implantación de calidad de manera que antes de comenzar el desarrollo sean conocidas por todo el equipo.
- Hay una serie de requisitos no mencionados de manera explícita que forman parte de todos los proyectos de desarrollo.
- Se recomienda el uso de una metodología que contemple el aseguramiento de la calidad en todas sus fases.

No obstante, también es preciso aclarar que a lo largo de este tema abordaremos principalmente la calidad del propio producto SW. No lo haremos sobre otros aspectos relacionados con el proceso de desarrollo, más allá de algunas técnicas específicas. Así, no abordaremos la calidad del planteamiento del propio proyecto, entendido como la adecuación de la iniciativa o de inicio de este. Salvo por las técnicas descritas en relación con Métrica v3, no se abordará la gestión y priorización de la cartera de pedidos, estudios de viabilidad del proyecto, de otros productos a obtener, de las tecnologías ni de las herramientas seleccionadas para su desarrollo. Tal y como indica el título del tema nos centraremos en la calidad del SW. Tampoco nos focalizamos en métricas que intentan medir la productividad de los programadores de manera individual o colectiva, aunque haremos una breve referencia a ellas.

2. EL SW COMO PRODUCTO DE CALIDAD

El SW consiste en la codificación de instrucciones en lenguajes de programación formales para ser interpretados o ejecutados por máquinas. También se incluye en la definición de SW, ficheros de configuración y de definición de estructuras de datos asociados. Es un producto final inmaterial, pura información. No se puede medir su calidad por sus medidas físicas, su resistencia o durabilidad. No se “estropea” ni se deteriora con el paso del tiempo, ni evoluciona en ningún sentido. La estructura de datos almacenados y recuperados por el SW también resultan críticos desde el punto de vista de la calidad y se deben analizar en este ámbito.

La medición de la calidad se debe realizar tanto como por el producto obtenido, en conjunción con su entorno de ejecución, por el cumplimiento de las funcionalidades requeridas, en unos tiempos de respuesta razonables y de consumo de recursos adecuados. Por ello, su calidad percibida depende, además, de otros factores de configuración y despliegue, ajenos al propio SW desarrollado, lo cual además dificulta su valoración.

El SW depende del equipamiento físico o hardware, HW, y con otro SW de base que usa o sobre el que se ejecuta. En este contexto sí que puede evolucionar (cuando cambian las versiones o la capacidad de su equipamiento) y además se requerirá que así sea para permitir las mejoras evolutivas de estas capas. Igualmente puede dejar de brindar el servicio esperado por un cambio en alguna de estas capas de servicio o de infraestructura que usa.

Los errores de SW provienen de su definición y, en algunos casos, solo se reproducen estos errores de manera puntual, intermitente y bajo ciertas condiciones de explotación, operación, o simplemente por una entrada de datos inesperada.

En algunos casos el producto a desarrollar es básicamente único, para un objetivo específico y un conjunto limitado de usuarios. En estos casos, no se dispone de antecedentes de la funcionalidad a alcanzar, que inicialmente sólo es imaginada por los usuarios demandantes del sistema. Esto supone que se requiere el seguimiento de controles de calidad específicos para este producto y que estarán asociados al cumplimiento unívoco de dichos requisitos. De hecho, muchas de las metodologías de pruebas abogan por la definición de pruebas al mismo tiempo que se establece la especificación inicial.

Nuestros ciudadanos y funcionarios usan cada día un sin fin de sistemas informáticos, tanto en la esfera laboral como en la personal. Por ello, aunque no se requieran explícitamente hay una serie de criterios de calidad que se consideran comunes a cualquier sistema nuevo a desarrollar. Entre ellos se pueden mencionar mecanismos adecuados de rendimiento, con tiempo de respuesta lógicos (que se consideran siempre inferiores a pocos segundos), de accesibilidad desde cualquier dispositivo, seguridad y control de accesos, de autenticación, de disponibilidad, etc.

El gran principio asociado a la calidad del SW es la simplicidad. Veremos métricas de calidad asociadas a la complejidad, que debemos mantener bajos número contenidos. Además, ante cualquier opción de diseño o de implementación, la solución más eficiente, eficaz y en definitiva de mayor calidad es siempre la más simple. Debemos entender como simple, la más sencilla de documentar, la que requiere de menos espacio para comentar, la que resulta más fácil de explicar a un compañero de trabajo, etc.

La calidad se puede estudiar desde dos puntos de vista diferentes: del propio del equipo de desarrollo y/o de un equipo externo que controle los productos obtenidos. Una organización madura en el desarrollo de SW ha conseguido desplazar el esfuerzo desde el grupo de control al propio equipo de desarrollo. Así la calidad se constituye en un patrón común e integrado en todos los proyectos, minimizando los controles de equipos externos y permitiendo que los equipos especializados de calidad se dediquen a la creación de estándares, de herramientas, mediciones

automatizadas y de mejora continua. En cambio, en una entidad inicial en el desarrollo de sistemas, la calidad se identifica como un lastre o un lujo, ajeno a los equipos de desarrollo, impuesta por la dirección, que supone un retraso en las entregas de funcionalidades. En esta evolución, inicialmente, se considera el proceso de aseguramiento de la calidad unos filtros que detectan problemas a un mecanismo que posteriormente previenen problemas más graves. Actualmente las organizaciones más punteras la consideran un factor estratégico, un elemento competitivo.

El control de calidad se posiciona en la defensa del cliente desde el interior de cada proyecto de desarrollo.

3. ESTÁNDARES NACIONALES E INTERNACIONALES

A principios de los años 80"s el Departamento de Defensa (DoD) de los Estados Unidos enfocó sus tareas a la revisión de los problemas del software y a su mejora. Para ello, a finales de 1984, se creó el Instituto de Ingeniería de Software (SEI, *Software Engineering Institute*) de la *Universidad Carnegie Mellon*, como Centro de Investigación y Desarrollo con la misión de liderar los avances para la mejora de la calidad de los sistemas dependientes del software.

Después de varios años de realizar cuestionarios, evaluaciones, consultas e investigaciones, en 1991 el SEI, junto a otras organizaciones, produce la primera versión del Modelo de Capacidad y Madurez del Software (SW-CMM, *Software Capability Maturity Model*).

Por otra parte, ISO (*Internacional Organization for Standardization*) plasma los estándares de calidad y desarrollo en 1987 con la norma ISO 9000, un conjunto de estándares internacionales para sistemas de calidad; en particular ISO 9001, y su complemento ISO 9000-3, son aplicables al proceso software y a organizaciones de desarrollo software.

Durante el mismo intervalo de tiempo, la Comisión Europea esponsorizó un proyecto llamado *BOOTSTRAP*, con el objetivo de acelerar la aplicación de técnicas de ingeniería del software a la industria del software europea.

Por su parte, en junio de 1989, ISO e IEC crean un *Joint Technical Committee* (JTC1) en Tecnologías de la Información. La metodología Bootstrap se encuentra alineada con la norma ISO 9000.

TRILLIUM es, a su vez, un modelo desarrollado por la empresa *Bell Canada* para valorar el proceso de creación del software de suministradores potenciales, minimizar los riesgos propios y asegurarlos tiempos de entrega de los productos.

El proyecto SPICE (*Software Process Improvement and Capability Determination*) promovido por la ISO (ISO/IEC 15504), es una actividad del WG 10 del Subcomité 7 del ISO/IEC JTC1. En 1991 se aprueba el estudio para obtener un estándar internacional para evaluación de procesos de desarrollo software con un marco de trabajo uniforme para gestión e ingeniería del software.

A partir del nuevo milenio, los enfoques ágiles como *Scrum* y *Extreme Programming* (XP), con *Agile* y *Scrum* ganaron popularidad en la industria del software. Si bien no son estándares en el sentido tradicional, estas metodologías se centran en la colaboración, la adaptación continua y la entrega de software de alta calidad de manera incremental. Scrum, en particular, se ha convertido en un marco de referencia ampliamente utilizado para el desarrollo ágil de software.

También debemos considerar el ISO/IEC 12207: Esta norma, publicada inicialmente en 2008, y revisada en 2017 se centra en los procesos de ciclo de vida del software y proporciona un conjunto de directrices y procesos para el desarrollo, la adquisición y el mantenimiento de sistemas y software. Es una norma importante para las organizaciones que buscan establecer procesos sólidos en todo el ciclo de vida del software.

La serie de normas, ISO/IEC 29110, publicada en 2011 y revisada en 2016, se desarrolló específicamente para pequeñas y medianas empresas (PYMES). Proporciona pautas y procesos simplificados basados en las mejores prácticas de ISO/IEC 12207 y ISO/IEC 15504 para ayudar a las PYMES a mejorar la calidad y la gestión de sus procesos de desarrollo de software.

A medida que la colaboración entre equipos de desarrollo y operaciones se volvió más estrecha, el movimiento DevOps se constituyó como su pieza angular. Aunque no es un estándar en sí mismo,

DevOps enfatiza la automatización, la colaboración y la entrega continua para mejorar la calidad y la eficiencia en el ciclo de vida del software.

Publicada en 2014, ISO/IEC 25000 (*SQuaRE System and Software Quality Requirements and Evaluation*) es organizar, enriquecer y unificar las series que cubren dos procesos principales: especificación de requisitos de calidad del software y evaluación de la calidad del software, soportada por el proceso de medición de calidad del software. Proporciona un marco para la evaluación de la calidad del software y la evaluación de productos de software. Se basa en la caracterización de la calidad del software y proporciona una estructura para medir la calidad en múltiples dimensiones, como funcionalidad, confiabilidad, usabilidad, eficiencia y más.

Estos estándares y enfoques continúan influyendo en la industria del software en el siglo XXI, brindando a las organizaciones herramientas y guías para mejorar la calidad y la eficiencia en el desarrollo de software en un entorno en constante evolución.

4. MÉTRICAS Y EVALUACIÓN DE LA CALIDAD DEL SOFTWARE

Métrica es una medida (cuantitativa) del grado, cantidad o tamaño que un producto o un proceso ofrecen. En el ámbito de desarrollo de SW y control de calidad una métrica es cualquier medida o conjunto de medidas que permiten caracterizar un software o sistema de información. Se aplican métricas que procuran mejorar la inversión, el porcentaje de retorno, optimizando el tiempo invertido en el desarrollo del producto y reduciendo sus costes. Pero esencialmente estas métricas nos ayudarán a medir el funcionamiento de nuestro código, así como su facilidad de mantenimiento.

Además, al aplicar métricas de software hacemos explícito nuestro producto y el estado de nuestro proyecto, prevemos fallos, reducimos costes y, junto con una descomposición funcional, mejoramos la estimación de tiempos que restan hasta el producto final. Generar SW de calidad reduce costes de mantenimiento. Las métricas son indispensables para objetivar estos avances, bajo el principio de lo que no se puede medir, no se puede gestionar, ni mejorar.

A lo largo de este apartado primero revisaremos unos conceptos previos para pasar después a una enumeración clasificada de las diferentes métricas definidas.

4.1 Definiciones previas

Acoplamiento (*coupling*)

El acoplamiento hace referencia a la dependencia o conectividad de un módulo o clase con otros módulos o clases. El objetivo en cualquier proceso de desarrollo es mantener esta dependencia o acoplamiento entre módulos lo más bajo posible. Así se generan unidades independientes fácilmente entendibles, mantenibles y extensibles de manera separada. Una modificación o cambio en uno de ellos no representa ningún cambio en otro.

Un módulo debe ser capaz de interactuar con otro a través de una interfaz estable y sin depender de otros, para su correcta implementación. Por el contrario, un sistema con un nivel de acoplamiento alto tendrá módulos que se encontrarán ligados entre sí y se comunicarán directamente unos con otros. Un código acoplado será difícil de comprender, reutilizar, y al no estar aislados, se deben considerar todas sus dependencias. Un problema común con este tipo de aplicaciones es que, en caso de un cambio en algún módulo, puede desencadenar un efecto dominó en otros módulos, por lo que será necesario rehacer código en el resto de los módulos dependientes.

Un código acoplado va directamente en contra del principio «*Tell, Don't Ask*» (TDA), (en castellano: indica o define y no preguntes) que promueve el encapsulado conjunto de los datos y de las funciones que operan sobre estos datos.

Cohesión (*cohesion*)

Cohesión define el grado de relación que existe entre los datos y métodos o funciones internos de un módulo. Debe ser lo más alta posible. Las diferentes partes de un módulo (constantes definidas, miembros o variables y métodos o funciones asociadas), deben tener una relación lógica lo más estrecha posible y a su vez con el menor acoplamiento o relación con el resto de módulos o clases.

Un módulo que siga el Principio de Responsabilidad Única o SRP por sus siglas en inglés, debe realizar una única cosa. Es muy habitual, si no prestamos atención a esto, que acabemos teniendo clases que tienen varias responsabilidades lógicas a la vez.

Complejidad (*complexity*)

Hace referencia a la dificultad de la lógica que sigue un determinado algoritmo.

Una de las métricas de calidad de software más importantes con la que contamos es la complejidad ciclomática, que puede ser usada en las fases de desarrollo o mantenimiento entre otras.

Esta métrica de software, propuesta por Thomas McCabe en 1976, se basa en el diagrama de flujo determinado por la estructura de flujo de ejecución de un determinado código. Del análisis de esta estructura se obtendrán las medidas cuantitativas que nos facilitarán la comprensión y mejora de estas. Cuanto más complejo es ese flujo, mayor será la complejidad. La complejidad ciclomática se define como el número de áreas separadas en un diagrama de secuencia que represente al algoritmo.

Resulta evidente una correlación directa entre la complejidad ciclomática y la cantidad de errores o bugs de un trozo de código, o el número de líneas de este. Por ello, podemos concluir que, a mayor complejidad y dimensión, se presentan mayores problemas y fallos en nuestro código.

También se puede hablar de complejidad de manera más formal o matemática como número máximo de ejecuciones individuales de un algoritmo, que se incrementan con el número de bucles y de condiciones que se anidan. También se denomina complejidad computacional. Se trata de un concepto similar a la complejidad ciclomática.

Se relaciona claramente con la complejidad cognitiva, con la claridad y facilidad de comprensión. Mide pues cuánto es de difícil entender intuitivamente un bloque de código, a diferencia de la complejidad ciclomática, que determina qué dificultad tiene recorrer todas las ramas o probar el código.

Existen diferentes prácticas que ayudan a reducir esta complejidad cognitiva, como la separación de partes internas en otras funciones o métodos especializados. Los desarrolladores encargados del mantenimiento agradecerán esta reducción de la complejidad para reducir los tiempos de las correcciones.

Cambios de códigos (*Code Churn*)

Es la cantidad de veces en la que el código ha sido modificado o la frecuencia con la que se añade, quita o altera el código a través del tiempo.

Esta métrica de calidad presenta una relación directa con código defectuoso. Cuantas más modificaciones sufra un código, mayor es la posibilidad de introducir un defecto o bug o de que sigan encontrando nuevos problemas en dicho código.

Esta métrica es fácil de calcular usando un sistema de control de versiones (SVSN subversion, git, mercurial, etc.). Basta contabilizar el número de “*commits*” (propuestas de subida de nueva versión al repositorio de código) que modificaron un dato, fichero, o toda una clase.

Código cubierto (*Code Coverage*)

Mide el porcentaje de código que se encuentra testeado por alguna prueba. Si un trozo de código se ha probado, tiene menos probabilidades de incluir algún error. Si no se ha probado, es más probable que cuando se den sus condiciones de ejecución, no se ejecute adecuadamente.

Tener test de calidad en nuestro proyecto que cubran el mayor porcentaje de código, ayudará a incrementar el valor de esta métrica de calidad de software y, a su vez, será menos probable que el código contenga bugs.

Debemos pues realizar todas las pruebas que aseguren que se ejecutan todas las ramas de dichas condicionantes (*if*) o bifurcaciones del programa, al menos una vez.

Código muerto (*dead code*),

El código muerto es código que no puede ser ejecutado con ninguna condición de entrada o de datos. Pueden darse condiciones imposibles de cumplir; las instrucciones bajo dichas condiciones son siempre código muerto.

En algunos casos puede ser difícil de identificar ya que, no siempre sabemos si cierto extracto de código se está ejecutando en producción o no, dependiendo de condiciones de entrada no previstas. Esta métrica de software es útil para verificar la calidad del código, sin embargo, no existe ningún método que detecte este código muerto y sea infalible. Por ejemplo, puede haber código que solo es ejecutado por herramientas o servicios externos, y que se encuentran fuera del control de herramientas que determinan si el código es utilizado o no. De ese modo se pueden generar falsos positivos.

Se resuelve eliminando el código muerto, una vez asegurado que no se ejecuta en ninguna condición. Tener pruebas automatizadas nos ayudarán a detectar potenciales errores al borrar código.

También se puede considerar código muerto el código comentado que se usa para otros entornos de despliegue.

Código duplicado (*code duplication*)

Código duplicado es el término utilizado para designar una serie de instrucciones de código que se declara (se repite) más de una vez dentro de una aplicación. En lugar de definir una función o método que incluya una única vez dicho código y sea llamada desde donde se requiera, estas duplicaciones suponen exponer el código a más errores y que las modificaciones requieren de mayor esfuerzo.

La existencia del código duplicado ocurre frecuentemente cuando el programador no está familiarizado con el código previamente desarrollado, lo que lo lleva a replicar trozos de código. El famoso: Ctrl C, Ctrl V.

Es importante entender que código duplicado no es sólo una copia de código exactamente igual, sino que puede ser código con ligeras adaptaciones, que se deberían declarar con procedimientos o funciones con parámetros.

Probablemente, el mayor enemigo del código limpio es el código duplicado. El principio DRY (*Don't Repeat Yourself*) debe ser aplicado.

La existencia de código duplicado afecta a los costos del proceso, ya que provocará una mayor complejidad ciclomática, un mayor número de pruebas unitarias aumenta innecesariamente el tamaño del proyecto, y resulta difícil de corregir. Esta corrección supone no solo declarar la funcionalidad de manera separada, sino además implica cambiar todas las copias.

Código dudoso (*dodgy code*, *bad code*)

Código que genera dudas respecto de su adecuación. Se trata de una de las etiquetas con las que las herramientas de análisis de código estático señalan algunas instrucciones que requieren de un análisis manual por parte del desarrollador para asegurar que son correctas.

Términos sobre rendimiento de algoritmos

El rendimiento es la cantidad de unidades de datos que procesa un sistema en un cierto período de tiempo. Por su parte, el tiempo de respuesta mide cuánto tiempo tarda un sistema en responder a una consulta o demanda.

La fiabilidad, disponibilidad y capacidad de servicio (RAS) se refiere a la capacidad del software para cumplir constantemente con sus especificaciones; cuánto tiempo funciona en relación con la cantidad esperada; y con qué facilidad se puede reparar o mantener.

Términos sobre corrección de errores

El tiempo medio de detección promedio que indica cuánto tiempo los usuarios del sistema tardan en detectar o notar un problema o error.

El tiempo medio entre fallos, resulta efectivamente el periodo medio entre paradas no deseadas del sistema. Esta métrica es un cálculo de cómo de común es que un programa o sistema falle. Es de esperar que a lo largo de la vida del sistema de información este tiempo se vaya incrementando.

El tiempo medio de diagnóstico es la media de tiempo requerido para encontrar el motivo de un fallo detectado, es decir, para determinar el módulo y función en la que se da el problema.

Por último, el tiempo medio de reparación es el promedio desde la detección a su corrección y puesta de nuevo en servicio, y que representa la rapidez con que un equipo aborda las fallas detectadas y que incluye los tiempos anteriores.

4.2 Métricas sobre el producto analizado

Las medidas sobre la calidad se pueden realizar sobre diferentes objetos conocidos como SW. Así atendiendo al producto analizado, se definen métricas:

4.2.1 Sobre documentación asociada

La metodología seguida en el desarrollo nos indicará los documentos a elaborar en cada fase que ella misma define. Así, siempre queda regulada la necesidad de aportar documentos de análisis o de diseño como fruto de esas fases. Por ello, primero debemos referirnos a la calidad de estos productos. El control de calidad supondrá fijarnos básicamente en si cumple la función para las que están previstos.

Los documentos de análisis sirven inicialmente para asegurar que los usuarios describen el sistema a implementar y que el producto a obtener será el requerido. Los documentos de diseño sirven para detallar los requisitos en criterios técnicos que puedan ser implementados por los programadores del proyecto. Igualmente habrá previstas guías de uso, de instalación, de formación, etc. con funciones específicas.

Debemos asegurar que cada documento cumple con su función esperada y registra los elementos clave de aseguran la terminación de cada etapa o función. Igualmente, que tenga la estructura, contenido y el tamaño adecuado y previsto en dicha metodología.

En general también debemos revisar que cada documento está bien versionado, es completo y se ha distribuido y aprobado por todos los responsables del mismo. Debemos comprobar su fecha de última actualización, si tiene contenido actualizado con cada modificación. También si presenta el índice adecuado o previsto y detalla cada apartado hasta un nivel correcto.

En cada caso, hay que asegurar que su contenido es correcto, en el sentido de que generan otros productos sin incidencias, suponen guías exactas y comprobadas de instalación, las guías de uso y material de formación corresponden a la versión asociada, incluyen ejemplos completos, etc.

Las modificaciones de los documentos iniciales de análisis de requisitos, arquitectura, diseño...), en fases avanzadas del proyecto suponen a su vez una métrica importante para valorar la calidad de las primeras etapas del desarrollo.

4.2.2 De código fuente

En este apartado debemos garantizar que se cumplen criterios puros de sintaxis, que el código entregado es el último, que permite la reconstrucción del sistema completo (su función última).

Dentro del código además se deberá realizar revisiones para controlar otros parámetros de calidad: la existencia de comentarios adecuados entre el código que permitan la comprensión de la funcionalidad de cada apartado por nuevos desarrolladores.

En cuanto a los comentarios, en determinados lenguajes de programación, como java, los comentarios se convierten en la propia documentación de la clase o paquete asociado, por lo que cobran especial interés. También se deben estudiar las instrucciones de pre-procesadores (como en el caso del lenguaje C), o el adecuado uso de las aserciones que ofrezca el lenguaje.

Resulta imprescindible garantizar convenciones de nomenclatura, estilo de codificación, indentación (uso de tabuladores para representar el nivel de profundidad en bucles de cada línea de código), etc. a lo largo de todos los módulos del proyecto, lo que independiza cada fuente de su autor específico.

Igualmente se debe controlar la nomenclatura usada para cada tipo de componente (paquetes en minúsculas, clases en mayúsculas, métodos en minúscula, variables, contadores, etc.), así como que dicha identificación resulta evidente con su función. Del mismo modo se debe asegurar la limpieza del código (evitar código muerto o comentado), etc.

La configuración debe estar siempre separada del código, de manera que cada nuevo despliegue suponga la modificación de estos ficheros de configuración claros y definidos, pero en ningún caso la modificación del código y su posterior compilación o traducción y empaquetado.

Por último y respecto del código se deben tener en cuenta la declaración de constantes, así como limitar el uso de constantes fijas de tipo cadena (*String*) o numéricos en el código, sin una declaración de constante única asociada.

4.2.3 De diseño

En este apartado consideramos diseño diferentes aspectos del SW desarrollado. Primero, entendido como el proceso de separación de trozos de códigos de un tamaño asimilable como funciones o métodos (sin repetición de código, o Ctrl-C Ctrl-V) con complejidad controlada y sin código muerto (no accesible).

También consideramos un diseño adecuado la agrupación de datos encapsulados (o en acoplados) con los métodos asociados en objetos de una determinada clase y de un tamaño adecuado de clases. Cada clase debe contener aquellos datos atributos que son comunes a la mayoría de los métodos incluidos. Los métodos deben tener el número mínimo de parámetros específicos.

En el apartado de diseño debemos considerar las relaciones de inclusión de objetos unos dentro de otros o la relación de herencia de unas clases respecto de otras. Si se usa un lenguaje estructurado no basado en objetos, debemos cuidar que las funciones se agrupan en módulos cohesionados, y no acoplados al resto.

En determinados lenguajes (java, c#, etc.) orientados a clases e interfaces (clases vacías o especificaciones de clases) dentro del diseño debemos considerar la exhaustiva definición de estos interfaces, de tal manera que todas las clases implementen al menos algún interfaz, que permita a estos objetos ser tratados de manera genérica por sus clases usuarias. En este apartado de diseño debemos asegurar igualmente la definición de objetos abstractos (o de implementación parcial).

Igualmente debemos considerar si las relaciones de herencia entre objetos cumplen con las descripciones funcionales facilitadas por los usuarios. También debemos considerar la agrupación de clases relacionadas en paquetes, de manera que agrupemos clases con una utilidad similar en nuestro programa.

Por último, debemos considerar el uso de patrones de diseño, de creación (de objetos por factorías, etc.) de comportamiento y estructurales, que revisaremos en un apartado posterior.

4.2.4 De arquitectura

En este apartado deberemos considerar la definición de un conjunto estable y común de paquetes en cada aplicación y la distribución de funciones entre ellas, según los estándares de la organización usuaria.

En general se debe considerar la arquitectura inicialmente prescrita: el número de capas de la aplicación y respetar la funcionalidad de cada una de ellas: capa de interfaz de usuario, capa de negocio o capa de acceso a datos, capa de acceso a servicios, capa de servicio propio de la aplicación, etc.

También se deberá asegurar que las funciones previstas se realizan en cada capa: la visualización en el HTML y el código de script de cliente (*javascript*) de cliente, que el acceso a los datos, se hacen a través de los procesos definidos y autenticados, etc.

Igualmente debemos asegurar el uso de los *frameworks* indicados para cada capa. Así en un modelo de dos capas, se asegura que se usen Angular, React, Vue o equivalente en la parte cliente, y que se hagan despliegues de multi-páginas o aplicaciones de una única página: SPA (*Simple Page Applications*). Igualmente, que se use determinadas utilidades en el servidor (como *spring Boot* o versiones específicas del *frameworks* determinados de .Net), arquitecturas de microservicios, uso de “*hubs*”, orquestadores, de colas de mensajería asíncrona, envío de correo electrónico, actualizaciones documentales, otros sistemas, etc.

En el cliente se deben considerar las guías de implementación de los *frameworks* usados. En el lado del servidor, se suelen especificar previamente al inicio del proyecto y hay que controlar que la funcionalidad se estructura en paquetes específicos como controladores (asociados a casos de uso y a tecnologías específicas: http, etc.), servicios (independientes de las tecnologías de acceso o consumo y relacionados con operaciones atómicas del negocio), objetos de mapeo de entidades de datos (uno por entidad del diagrama entidad/relación), objetos de acceso a la base de datos o a sistemas documentales, etc.

También se deberá asegurar que aquellos aspectos definidos horizontales se tratan como tales y las utilidades usadas, así como los diseños: capa de autenticación, de autorización, capa de monitorización, etc. a través del uso de programación orientada a aspectos, etc.

Se puede igualmente considerar la calidad en cuanto al uso de sistemas compartidos en un entorno tecnológico específico. Así si el sistema se va a alojar como componente en un servicio *hyper-escalar*, (*Amazon Web Services*, *Google Cloud*, *Microsoft Azure* u otros) se debe considerar una arquitectura que haga uso de las funcionalidades previstas por dicho alojamiento: separación en contenedores (*docker*), sistemas de balanceo (*kubernetes*), uso de servicios (*functions*, etc.).

Por último, se relaciona con aspectos de calidad el uso de sistemas comunes o compartidos en cualquier organización y en el ámbito de los sistemas de gestión de procedimientos de ciudadanos de la AGE: sistemas de autenticación comunes a la AGE, envío de notificaciones, sedes electrónicas, etc.

4.3 Métricas de aspectos específicos

En este apartado vamos a clasificar las métricas en relación al aspecto que se desea asegurar:

- Métricas de preferencia: estilo de código, convenciones, limitaciones, etc. como las representadas en el apartado anterior, para maximizar la cohesión y reducir el acoplamiento.
- Métricas de complejidad: volumen, tamaño, anidaciones, bucles, y configuración. Debemos mantener todo nuestro código lo más simple posible.
- Métricas de desempeño: miden la conducta de módulos y sistemas de un software, bajo la supervisión del SO o hardware.
- Métricas de exactitud: miden la precisión y validez de cada clase o paquete, con pruebas individuales durante la fase de subida al repositorio y de integración continua durante el despliegue de nuevas versiones.
- Métricas de rendimiento: miden tiempos de ejecución del software, tanto de cada uno de sus módulos, como del sistema al completo. Se trata por ejemplo de velocidad de respuesta, capacidad de respuesta ante datos en los límites previstos, o ante datos erróneos; estabilidad (cambios de tiempo de respuesta en el tamaño de los datos de entrada) o disponibilidad (% de caídas o tiempo entre fallos)
- Métricas de eficiencia: uso de cada minimización de latencias, velocidad de respuesta, capacidad de gestión de bloques.
- Métricas de configuración: la configuración del entorno del sistema para su fácil y cómoda migración entre diferentes entornos.
- Métricas de usabilidad y de interfaz de usuario: ayudan a reducir los tiempos de uso y la formación inicial de los usuarios. Así como, es difícil clasificar las emociones de las personas, también es un desafío evaluar su reacción al software. Sin embargo, se pueden definir métricas para medir la facilidad para efectuar las tareas más habituales de los usuarios de nuestros sistemas: profundidad de la navegación, número de “clicks de ratón” para alcanzar las funcionalidades más usadas, cambios de contexto, necesidad de datos de otras ventanas... incluye otros criterios relacionados como la facilidad de descubrimiento de las funciones de un sistema. la facilidad de aprendizaje en su uso, la eficiencia en la navegación, etc.
- Métricas de accesibilidad. miden la posibilidad de operación del sistema en condiciones de acceso con limitaciones sensoriales, cognitivas o de comprensión de los usuarios.
- Métricas de recomendación: Net Promoter Score (NPS). Esta métrica de software refleja la voluntad de los clientes de recomendar una aplicación a otros. Net Promoter Score se presenta como un rango de números de 0 a 10. Los clientes con una puntuación de 0 a 6 son Detractores; las puntuaciones 7 y 8 son Pasivos; y 9 y 10 son Promotores.

- Métricas de defectos: densidad de defectos medidas por código y nivel de código. Los desarrolladores pueden contabilizar el número de defectos por KLOC (Kilo Líneas de Código) para evaluar la frecuencia de los defectos. Incluye también medidas de cobertura de código cubierto en pruebas. También se puede usar el porcentaje de detección de defectos: cantidad de defectos encontrados antes de los lanzamientos de software en comparación con el número encontrado después del lanzamiento. Es preferible un alto porcentaje, ya que eso significa que se encontró una mayor proporción de defectos antes de que los clientes usaran el software. También se define la deuda técnica: es una metáfora que refleja el esfuerzo a largo plazo, así como los costes temporales y financieros, de los problemas detectados en el código entregado para dejarlo perfecto según cualquier métrica.
- Métricas de seguridad: controlan que el uso de la aplicación no permite el acceso a otros recursos no permitidos o previstos, o a estos de forma no predefinida. Pueden incluir vulnerabilidades de seguridad, determinados a través de análisis de ataques y vulnerabilidades conocidas. También se pueden medir los incidentes de seguridad reales. Este KPI cuenta la cantidad de veces que un hacker aprovecha una vulnerabilidad en el software y la gravedad del ataque (por ejemplo, qué datos se robaron) así como la cantidad de tiempo que duró el incidente.
- Métricas en mantenimiento: Se pueden definir:
 - Coste de encontrar un defecto en las pruebas como la división entre el esfuerzo total gastado en las pruebas y los defectos encontrados en las pruebas.
 - Proporción de esfuerzo de reelaboración = $(\text{Esfuerzos reales de reelaboración gastados en esa fase} / \text{esfuerzos reales totales gastados en esa fase}) \times 100$.
 - Evolución del flujo de requisitos = $(\text{Número total de requisitos añadidos} / \text{Nº. de requisitos iniciales}) \times 100$.
 - Variación del cronograma = $(\text{Fecha real de entrega} - \text{Fecha planificada de entrega})$.
 - Desviación del cronograma = $(\text{Fecha de finalización real} - \text{Fecha de finalización estimada}) / (\text{Fecha de finalización planificada} - \text{Fecha de inicio planificada}) \times 100$.
- Métricas de prueba: Se pueden definir las siguientes:
 - Número de pruebas ejecutadas por período de tiempo = $\text{Número de pruebas ejecutadas} / \text{Tiempo total}$.
 - Eficiencia del diseño de pruebas = $\text{Número de pruebas diseñadas} / \text{Tiempo total}$.
 - Eficiencia de revisión de pruebas = $\text{Número de pruebas revisadas} / \text{Tiempo total}$.
 - Porcentaje de casos de prueba aprobados = $(\text{Número de pruebas aprobadas} / \text{Número total de pruebas ejecutadas}) \times 100$.
 - Porcentaje de casos de prueba fallidos = $(\text{Número de pruebas fallidas} / \text{Número total de pruebas ejecutadas}) \times 100$.
 - Porcentaje de casos de prueba bloqueados = $(\text{Número de pruebas bloqueadas} / \text{Número total de pruebas ejecutadas}) \times 100$.
 - Porcentaje de defectos reparados = $(\text{Defectos reparados} / \text{Defectos informados}) \times 100$.
 - Porcentaje de defectos aceptados = $(\text{Defectos aceptados como válidos por el equipo de desarrollo} / \text{Total de defectos informados}) \times 100$.
 - Porcentaje de defectos diferidos = $(\text{Defectos diferidos para versiones futuras} / \text{Total de defectos informados}) \times 100$.
 - Porcentaje de defectos críticos = $(\text{Defectos críticos} / \text{Defectos totales informados}) \times 100$.

4.4 Otras métricas del proceso de desarrollo

Existen muchas formas de discutir o evaluar la eficiencia del equipo y el trabajo completado. Las métricas de productividad permiten a los gerentes de desarrollo ejecutar mejor los proyectos. Entre ellas:

- Tiempo de entrega (lead time). El tiempo de entrega es el tiempo que tarda algo de principio a fin. En el desarrollo de software, por ejemplo, el tiempo de entrega de un proyecto comienza con la propuesta y termina con la entrega.
- Cantidad de código. Esta métrica de software, también llamada miles de líneas de código (KLOC), determina el tamaño de una aplicación. Si este indicador es alto, podría indicar que los desarrolladores fueron productivos en sus esfuerzos de programación. Sin embargo, esta métrica no es útil cuando un equipo de desarrollo intenta comparar dos proyectos escritos en diferentes lenguajes de programación. Además, tenga en cuenta que más código no siempre hace que el código sea eficiente o efectivo, lo que puede significar más trabajo de refactorización más adelante.
- Trabajo en curso (WIP, *Work In Progress*). En un contexto de ingeniería de software, WIP es un trabajo de desarrollo en el que el equipo ha comenzado a trabajar y que ya no está en el backlog. Un equipo puede expresar WIP en un gráfico de quemado. Una herramienta común para los “sprints” ágiles y Scrum, estos gráficos muestran cuánto trabajo ha terminado el equipo y la cantidad de trabajo que queda por hacer.
- Velocidad ágil. Para calcular la velocidad, un equipo de desarrollo de software ágil analiza los “sprints” anteriores y cuenta el número de historias de usuario o puntos de historia completados a lo largo del tiempo. La velocidad ágil es una estimación de qué tan productivo será el equipo en un solo sprint.
- Tasa de éxito de la meta del sprint. Esta métrica de software calcula el porcentaje de elementos que completó el equipo de desarrollo en el backlog del sprint. Es posible que un equipo no termine el 100 % del trabajo durante un sprint determinado. Sin embargo, el progreso del equipo aún podría cumplir con su definición de terminado: el umbral que debe alcanzar un proyecto para que una organización lo considere terminado. Si la iteración cumple con la definición de hecho, es un éxito.
- Número de versiones de software. Los equipos ágiles y de DevOps dan prioridad a los lanzamientos de software continuos y frecuentes. Con este indicador, los equipos pueden realizar un seguimiento de la frecuencia con la que lanzan software, ya sea mensual, semanal, diaria, por hora o en cualquier otro período de tiempo, y si esa cadencia finalmente ofrece suficiente valor comercial.

A lo largo de este apartado se han presentado primero unos conceptos previos, para después señalar las principales métricas en base al producto y a la fase analizada. Después se han presentado otras métricas específicas del producto SW, en base a aspectos propios, para finalizar con métricas dedicadas al proceso de desarrollo y no al producto final.

5. IMPLANTACIÓN DE LA FUNCION DE CALIDAD

La evaluación de la calidad del SW se propone como un proceso paralelo al propio proceso de desarrollo de este y se realiza a través de pruebas. Cada vez que se desarrolla un módulo, el mismo equipo de desarrollo debe proponer casos de prueba que se ejecuten automáticamente en cada despliegue. Además, en la integración de distintos módulos que se llaman unos a otros, se deben realizar otros tipos de pruebas de integración.

La ejecución de pruebas debe estar automatizada en el propio proceso de integración y despliegue de cada versión, de forma que cada vez que se proponga una nueva versión, se realicen automáticamente dichas pruebas durante el proceso de despliegue.

En este apartado a analizaremos principios de calidad del SW desde dos puntos de vista. Continuamos con los errores más comunes detectados en SW, así como pruebas de producto que nos permitirán su control o aseguramiento.

5.1 PRINCIPIOS DE CALIDAD

5.1.1 Principios SOLID

Principio de responsabilidad única

“A class should have one, and only one, reason to change.”

La S del acrónimo se refiere a *Single Responsibility Principle* (SRP). Según este principio “una clase debería tener una, y solo una, razón para cambiar”. Es esto, precisamente, “razón para cambiar”, lo que Robert C. Martin identifica como “responsabilidad”. Se enfrenta al diseño de clases con varias funciones, lo que complica la mantenibilidad posterior.

El principio de “Responsabilidad Única” es el más importante y fundamental de SOLID, muy sencillo de explicar, pero el más difícil de seguir en la práctica.

Para conseguirlo, deberemos agrupar juntos aquellos datos y procesos que cambien a la vez, por las mismas. Se puede igualmente expresar como el consejo de separar aquellas clases en otras que cambian por razones diferentes.

Principio de abierto/cerrado: *Open/Closed Principle (OCP)*

“You should be able to extend a classes behavior, without modifying it.”

El segundo principio de SOLID prescribe que una clase bien definida se debe poder extender o ampliar su comportamiento, sin modificarla. En otras palabras: diseñar las clases para resultar abiertas para poder extenderse, pero cerradas para modificarse.

Este principio resulta especialmente importante tener en cuenta el a la hora de desarrollar clases, librerías o *frameworks*, sin una aplicación directa o univoca a un proyecto.

Principio de sustitución de Liskov

“Derived classes must be substitutable for their base classes.”

La L de SOLID alude al apellido de quien definió este principio, Barbara Liskov, y nos prescribe que “las clases padre deben poder sustituirse por sus clases hijas de implementación”.

Esto significa que los objetos deben poder ser reemplazados por instancias de sus subtipos sin alterar el correcto funcionamiento del sistema o lo que es lo mismo: si en un programa utilizamos cierta clase, deberíamos poder usar cualquiera de sus subclases sin interferir en la funcionalidad del programa.

Según Robert C. Martin incumplir el *Liskov Substitution Principle* (LSP) implica violar en cierto sentido también el principio de Abierto/Cerrado.

Principio de segregación de la interfaz:

“Make fine grained interfaces that are client specific.”

El cuarto principio de SOLID sugiere definir interfaces que sean específicas para un tipo de cliente, es decir, para una finalidad concreta.

Definimos “interface”, como la definición de los métodos (con argumentos y tipos de respuesta) que se agrupan en un conjunto lógico. En este sentido, según este principio es preferible contar con muchas “interfaces” que definan pocos métodos que tener una “interface” forzada a implementar muchos métodos a los que no dará uso. Y además así, resultará más fácil de implementarse por las clases finales.

Principio de Inversión de Dependencias.

“Depend on abstractions, not on concretions.”

El último principio nos prescribe hacer clases que “dependan de abstracciones, no de clases concretas”. Los módulos de alto nivel no deberían depender de módulos de bajo nivel. Ambos deberían depender de abstracciones o, dicho de otra manera, las abstracciones no deberían depender de los detalles. Los detalles deberían depender de las abstracciones.

El objetivo de este principio consiste en reducir las dependencias entre los módulos del código, es decir, alcanzar un bajo acoplamiento de las clases.

5.1.2 Patrones de diseño

La calidad del SW se puede aproximar en base al uso de una serie de patrones de diseño ampliamente usados y recomendados por los mejores diseñadores y programadores. Por ello en este apartado vamos a realizar un breve repaso de las recomendaciones de estos patrones:

Patrones de creación

Se basan en evitar la codificación de la creación “new” en cada cliente de un tipo de objetos, el tipo de clase que lo implementa, sino delegar esta decisión a aspectos de configuración que no requieran una re-compilación o a otras clases para definir una única instancia. Tipos:

- **Factory Method.** Define una interfaz para crear un objeto, pero deja que sean las subclases quienes decidan qué clase instanciar. Permite que una clase delegue en sus subclases la creación de objetos.
- **Abstract Factory.** Proporciona una interfaz para crear clases de objetos, sin especificar sus clases concretas.
- **Builder.** Separa la construcción de un objeto complejo de su representación, de forma que el mismo proceso de construcción pueda crear diferentes representaciones.
- **Prototype.** Especifica los tipos de objetos a crear por medio de una instancia prototípica, y crear nuevos objetos copiando este prototipo.

- Singleton. Garantiza que una clase sólo tenga una instancia, y proporciona un punto de acceso global a ella.

Patrones de comportamiento

Definen tipos de objetos útiles para desacoplar funcionamientos específicos:

- Chain of Responsibility. Evita acoplar el emisor de una petición a su receptor, al dar a más de un objeto la posibilidad de responder a la petición. Crea una cadena con los objetos receptores y pasa la petición a través de la cadena hasta que esta sea tratada por algún objeto.
- Command. Encapsula una petición en un objeto, permitiendo así parametrizar a los clientes con distintas peticiones, encolar o llevar un registro de las peticiones y poder deshacer las operaciones.
- Interpreter. Dado un lenguaje, define una representación de su gramática junto con un intérprete que usa dicha representación para interpretar las sentencias del lenguaje.
- Iterator. Proporciona un modo de acceder secuencialmente a los elementos de un objeto agregado sin exponer su representación interna.
- Mediator. Define un objeto que encapsula cómo interactúan un conjunto de objetos. Promueve un bajo acoplamiento al evitar que los objetos se refieran unos a otros explícitamente, y permite variar la interacción entre ellos de forma independiente.
- Memento. Representa y externaliza el estado interno de un objeto sin violar la encapsulación, de forma que éste puede volver a dicho estado más tarde.
- Observer. Define una dependencia de uno-a-muchos entre objetos, de forma que cuando un objeto cambia de estado se notifica y actualizan automáticamente todos los objetos.
- State. Permite que un objeto modifique su comportamiento cada vez que cambia su estado interno. Parecerá que cambia la clase del objeto.
- Strategy. Define una familia de algoritmos, encapsula uno de ellos y los hace intercambiables. Permite que un algoritmo varíe independientemente de los clientes que lo usan.
- Template Method. Define en una operación el esqueleto de un algoritmo, delegando en las subclasses algunos de sus pasos. Permite que las subclasses redefinan ciertos pasos del algoritmo sin cambiar su estructura.
- Visitor. Representa una operación sobre los elementos de una estructura de objetos. Permite definir una nueva operación sin cambiar las clases de los elementos sobre los que opera.

Patrones estructurales

Permiten la comunicación de clases que presentan diferentes interfaces:

- Adapter. Convierte la interfaz de una clase en otra distinta que es la que esperan los clientes. Permiten que cooperen clases que de otra manera no podrían por tener interfaces incompatibles.
- Bridge. Desvincula una abstracción de su implementación, de manera que ambas puedan variar de forma independiente.
- Composite. Combina objetos en estructuras de árbol para representar jerarquías de parte-todo. Permite que los clientes traten de manera uniforme a los objetos individuales y a los compuestos.
- Decorator. Añade dinámicamente nuevas responsabilidades a un objeto, proporcionando una alternativa flexible a la herencia para extender la funcionalidad.

- Facade. Proporciona una interfaz unificada para un conjunto de interfaces de un subsistema. Define una interfaz de alto nivel que hace que el subsistema sea más fácil de usar.
- Proxy. Proporciona un sustituto o representante de otro objeto para controlar el acceso a éste.

Un ejemplo de *frameworks* que usa algunos de estos principios (*singletons* o factorías) junto con los de programación orientada a aspectos (prototipos y proxys de objetos) en lenguaje java es “*Spring*”, ampliamente usado en dicho entorno java y servidores asociados: *tomcat*, etc.

5.2 Errores del software más comunes

Hay varios portales públicos que condensan de manera actualizada los errores más habituales detectados en sistemas de información, incluyendo con códigos ejemplos para identificar e incluso explotar las vulnerabilidades asociadas. Se dan especialmente en el caso de errores de seguridad, pero las propias herramientas de análisis de calidad (*findbugs*, *PMD*, *checkStyle*, etc.) de código estructuran sus tipos de errores detectados en estos mismos conjuntos.

Estos errores se pueden clasificar en los siguientes casos:

- Escaso control y análisis de las entradas a nuestros programas, de forma que dichas entradas son usadas como parte de comandos SQL de acceso o incluso de actualización (con la posibilidad de acceso descontrolado a la base de datos) o para generar otra página o fragmento de página web (entrada de un blog), por lo que se da control a los usuarios para que generen código oculto en dichas páginas.
- Acceso a zonas de memoria no controladas. El uso de lenguajes de programación que controlan el acceso a memoria minimiza este tipo de errores, no obstante, los lenguajes de bajo nivel, con acceso directo por operar con punteros, pueden tanto acceder a zonas de memoria no previstas (fuera de los límites de las estructuras de datos reservadas) como zonas de programas, otros datos, otros procesos, etc.
- Consumo excesivo de recursos. Se pueden dar errores en la liberación de recursos (amplios espacios de memoria, descriptores de ficheros, sockets de conexión, conexiones de bases de datos, etc.) que suponen el consumo y agotamiento de recursos del sistema operativo. Esto a su vez puede suponer pérdidas significativas de rendimiento o incluso paradas no controladas.

Posteriormente analizaremos las herramientas citadas en el apartado 7.

5.3 Pruebas de SW

Si queremos detectar los problemas de calidad de un software antes de su puesta en producción debemos implementar una metodología de pruebas exhaustiva.

Aunque hay temas específicos para la preparación sobre pruebas de SW, hacemos un breve repaso de los tipos de pruebas que aseguran la calidad del SW por diferentes criterios:

- Grado de conocimiento del entorno y del sistema a probar: de caja blanca, negra y gris o semi transparente.
- Momento del desarrollo asociado: unitarios, de integración de subsistemas o de aceptación del sistema completo.
- Objetivo perseguido: pruebas de exactitud, de rendimiento, de seguridad, etc. de manera paralela a las métricas previamente definidas.

En este apartado se han presentado la implantación de la función de calidad, a través del análisis de los principios de diseño que garantizan el SW de calidad, como son los principios SOLID y los patrones de diseño de SW. Desde otro punto de vista, se han analizado los errores más comunes o frecuentes detectados en el SW actual. Por ultimo se han presentado las pruebas de aseguramiento de calidad, aunque no se han desarrollado, por tratarse más detalladamente en el tema específico: “96. Procesos de pruebas y garantía de calidad en el desarrollo de software. Planificación, estrategia de pruebas y estándares. Niveles, técnicas y herramientas de pruebas de software. Criterios de aceptación de software.”

6. LA IMPLANTACIÓN DE LA FUNCIÓN DE CALIDAD

Este apartado lo estudiaremos desde dos puntos de vista. Primero abordaremos la constitución de estándares y normas y de un equipo específico de gestión de calidad. Después abordaremos la implantación de calidad en cada proyecto específico, siguiendo básicamente las recomendaciones de la Metodología Métrica 3.

Existen varias aproximaciones a la implantación de un proyecto de mejora de la calidad del SW desarrollado. Desde el comienzo de este proceso se debe considerar que se trata de un trabajo evolutivo que, como cualquier sistema de mejora continua, no tiene un límite o meta final previamente especificado.

Podemos inicialmente optar por la constitución de un equipo específico de calidad. Dicho equipo puede iniciar su trabajo a través del análisis del código de cualquiera de los proyectos desplegados y de los que dispongamos de un servicio de mantenimiento. De dicho análisis se deberá compilar un conjunto de las primeras recomendaciones a los equipos de mantenimiento, de manera que se centren en los problemas de seguridad, de documentación y de rendimiento. Difícilmente se abordará un completo rediseño de un sistema a través de esta aproximación.

Pero este equipo de calidad puede igualmente desplegarse de otra manera. De forma previa al análisis de código de cualquier proyecto, es preciso definir previamente el conjunto de estándares objetivos, de normas a aplicar y de valores a alcanzar en cada métrica. Este equipo se debe encargar de centralizar y hacer público estos estándares y normas.

Igualmente, pueden responsabilizarse de la instalación y uso de un sistema de integración continua, que incluya pruebas básicas y controles en el proceso de despliegue.

Por último, este equipo puede planificar e impartir cursos de formación y concienciación de las ventajas de la implantación de controles de calidad, de técnicas, y herramientas a usar y de os métricas mínimas esperables en cada apartado.

Otro aspecto para prever en el despliegue de la función de calidad es la dependencia orgánica del equipo propuesto para esta función. Por un lado, va a controlar el trabajo de los equipos de desarrollo, por lo que no se debería dar una jerarquía de bajo nivel entre ellos; es más, defiende ciertos aspectos de fiabilidad, tiempo entre fallos, etc., que se suelen encomendar a los departamentos de sistemas, por lo que se puede considerar que actúan como defensores de la calidad de forma previa al paso a producción o a la operación del sistema.

No obstante, el conjunto de tecnologías que deben dominar el equipo de calidad está relacionado con aspectos de desarrollo, por lo que su dependencia del área de sistemas, tampoco suele generar ninguna sinergia.

Por ello parece recomendable que el equipo de calidad dependa directamente de la dirección TIC, como hermano de los equipos de desarrollo y de sistemas y que sirvan de filtro previo para el paso a producción de cualquier sistema nuevo a implantar.

6.1 Constitución de estándares, normas, etc.

Este apartado sobre la constitución de estándares resulta suficientemente amplio como para incluirse en algunos temas específicos del temario. No obstante, es bueno hacer una referencia aquí, para indicar que resultan imprescindibles para la implantación de una función de calidad de SW específica en cualquier unidad TIC.

Tal y como se han abordado en apartados previos, se requieren:

- Estándares de arquitectura: descripción del entorno de pruebas y de producción en el que deberá operar. Así como de los sistemas con los que deberá intercambiar datos. Descripción de las diferentes capas de los sistemas a desarrollar, definición de funciones o estructuración en paquetes de funcionalidad predefinidos, etc.
- Estándares de diseño: con el detalle de todos los aspectos que se requieren de manera común a cualquier sistema desarrollado por dicha unidad, tales como máximos de complejidad, de cohesión, mínimos de acoplamiento, tamaños de paquetes y de clases, uso de interfaces, etc.
- Estándares de código: que hagan independiente el código del propio proyecto y especialmente del autor del mismo. Entre estos se puede citar comentarios, nomenclaturas, estilos de codificación, documentación, etc.
- Estándares de configuración, de instalación, de operación, etc.

6.2 Constitución del equipo de gestión de calidad

El equipo de calidad debe ser formado por expertos desarrolladores. No puede resultar que el equipo de calidad disponga de menos conocimientos de arquitecturas modernas, entornos de despliegue, patrones y principios de diseño SW y especialmente de codificación que aquellos equipos cuyos productos va a evaluar.

De hecho, una de las técnicas más usadas en control de calidad son sesiones de revisión de código fuente. Un programador experto, con mucho tiempo de participación en proyecto con la misma tecnología es capaz en muy poco tiempo de evaluar una calidad inicial de código. Prácticamente de un vistazo revisa los principios de nomenclatura, comentarios, estructuración del código, identificadores, etc.

De ser posible, deberían seleccionarse para esta función aquellos desarrolladores que cuenten con experiencia y reconocido prestigio en la organización. Así nos aseguramos de que sus preceptos sean claramente asimilados y reconocidos como útiles y beneficiosos para el conjunto de la misma.

Inicialmente deben apoyar la definir o formalización de estándares claros, apoyar con formación de herramientas e interpretación de informes, integrar las mismas herramientas en el proceso de desarrollo que ejecutarán el control de calidad, revisar el resultado de las pruebas realizadas por el propio equipo de desarrollo.

Además, deben desarrollar ejemplos, capas comunes reutilizables, acceso a tablas comunes compartidas, etc. y ofrecerse como consultores previos en cualquiera de los nuevos proyectos a desarrollar.

También debemos definir si conviene enfocar el esfuerzo inicial de calidad a nuevos proyectos, para que así surjan de manera integrada con calidad desde el inicio. Esta aproximación no reduce los problemas de sistemas previos. Supone cambiar la forma de trabajo en el inicio de nuevo proyecto, lo que resulta más adecuado.

Por otro lado, se puede inicialmente revisar el SW de proyectos ya desplegados y previos a esta implantación. Esto nos asegura la remisión de su deuda técnica, y considerar la calidad como un aspecto de mejora en el mantenimiento evolutivo del sistema.

Como se pone de manifiesto cada enfoque tiene sus ventajas e inconvenientes. Además, ambas estrategias no son excluyentes y pueden ser utilizadas dependiendo de cada proyecto y de su estado de desarrollo.

6.3 Implantación de calidad por proyecto según METRICA

En el caso de que se haya decidido abordar la implantación de procesos de control de calidad en cada nuevo proyecto, ejecutaremos las tareas previstas por la metodología Métrica.

Primero analizamos el flujo de fases completo previsto en esta metodología, recogiendo en la primera línea las fases previstas y en la segunda la gestión del proyecto:

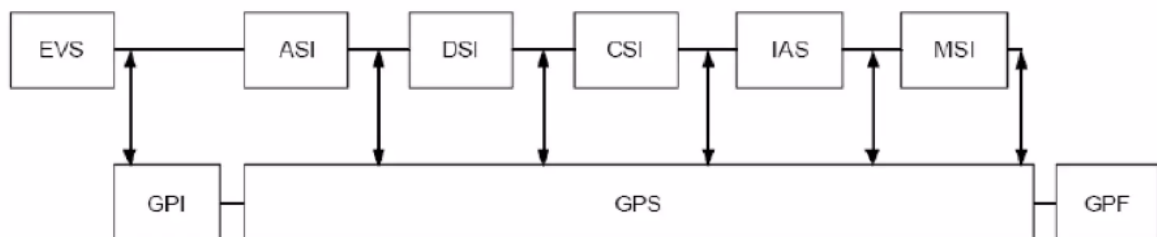


Ilustración 1. Fases de metodología METRICA v3

La estructura de MÉTRICA Versión 3 incluye un conjunto de interfaces que definen una serie de actividades de tipo organizativo o de soporte al proceso de desarrollo y a los productos, que en el caso de existir en la organización se deberán aplicar para enriquecer o influir en la ejecución de las actividades de los procesos principales de la metodología y que si no existen habrá que realizar para complementar y garantizar el éxito del proyecto desarrollado con MÉTRICA Versión 3. Uno de estos interfaces es el de CALIDAD.

El objetivo de la interfaz de Aseguramiento de la Calidad de MÉTRICA Versión 3 es proporcionar un marco común de referencia para la definición y puesta en marcha de planes específicos de aseguramiento de calidad aplicables a proyectos concretos. Las actividades propias de la interfaz de Calidad en MÉTRICA Versión 3 están orientadas a verificar la calidad de los productos. Son actividades que evalúan la calidad y que son realizadas por un grupo de Asesoramiento de la Calidad independiente de los responsables de la obtención de los productos.

Estas actividades de interfaz de MÉTRICA Versión 3 no entran en contradicción con el Plan General de Garantía de Calidad (PGGC), siendo lo suficientemente abiertas como para soportar una nueva versión del PGGC en el futuro.

Las actividades contempladas en la interfaz de Aseguramiento de la Calidad permitirán:

- Reducir, eliminar y prevenir las deficiencias de calidad de los productos a obtener.
- Alcanzar una razonable confianza en que las prestaciones y servicios esperados por el cliente o el usuario queden satisfechas.

6.3.1 ESTUDIO DE VIABILIDAD DEL SISTEMA

En este proceso el grupo de aseguramiento de calidad inicia el estudio de los sistemas de información definidos en cada alternativa de solución propuesta, con el fin de identificar las condiciones en que se van a desarrollar y/o a implantar, así como las características que deben reunir en cuanto a operación, mantenibilidad y portabilidad, para satisfacer las necesidades del cliente y los requisitos especificados.

La necesidad de establecer un plan específico de aseguramiento de calidad y el grado de intensidad con el que se aplican las actuaciones de calidad, vendrá determinada en función de este estudio y de los riesgos analizados por el equipo de desarrollo.

Una vez tomada la decisión de llevar a cabo un plan de aseguramiento de calidad en las alternativas propuestas, se define el contenido de dicho plan, de acuerdo con los estándares de calidad, si existen en la organización, sino se recomienda acudir a los estándares UNE-EN-ISO 9001:2000 Sistemas de Gestión de la Calidad – Requisitos y UNE-EN-ISO 9000:2000 Sistemas de Gestión de la Calidad – Fundamentos y vocabulario.

El plan de aseguramiento de calidad debe cubrir todas las necesidades establecidas de modo que, aquellas normas impuestas por los usuarios o clientes que difieran de las existentes en el sistema de calidad deben quedar también reflejadas en el plan.

El siguiente esquema muestra la correspondencia entre las actividades del proceso EVS y las de la interfaz de Aseguramiento de la Calidad.

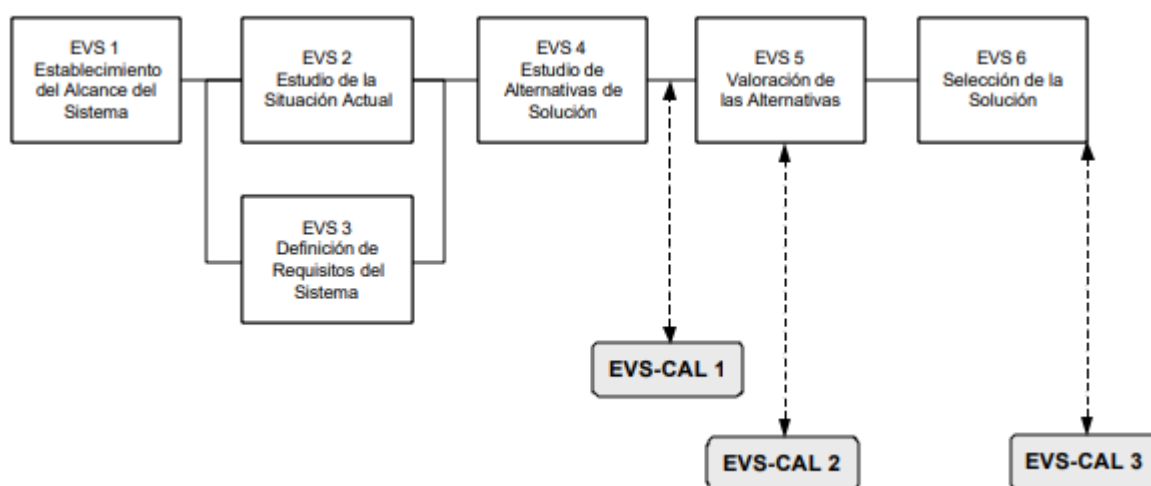


Ilustración 2. Controles de calidad en el estudio de viabilidad. METRICA v3

6.3.2 ANÁLISIS DEL SISTEMA DE INFORMACIÓN

En este proceso se define de forma detallada el plan de aseguramiento de calidad para un sistema de información, a partir de la especificación resultante del proceso Estudio de Viabilidad del Sistema (EVS). También se detallan los estándares y normas a cumplir, las revisiones a llevar a cabo y sobre qué productos, así como los procedimientos y mecanismos necesarios para resolver los problemas que surjan, definiendo las acciones preventivas o correctoras para su posterior corrección e identificando quiénes son los responsables en cada caso.

En el proceso Análisis del Sistema de Información (ASI), el grupo de aseguramiento de calidad se implica directamente en la revisión de los siguientes productos:

- Catálogo de requisitos, comprobando hasta qué punto se han definido de una forma que facilite su comprensión y seguimiento.
- Modelos resultantes del análisis, asegurando que se han verificado y validado y que se ha realizado la trazabilidad de requisitos.
- Plan de pruebas, comprobando que se han tenido en cuenta en su definición los criterios establecidos en el plan de aseguramiento de calidad, con el fin de facilitar en los procesos Diseño del Sistema de Información (DSI), Construcción del Sistema de Información (CSI)

e Implantación y Aceptación del Sistema (IAS) la revisión de los distintos niveles de prueba.

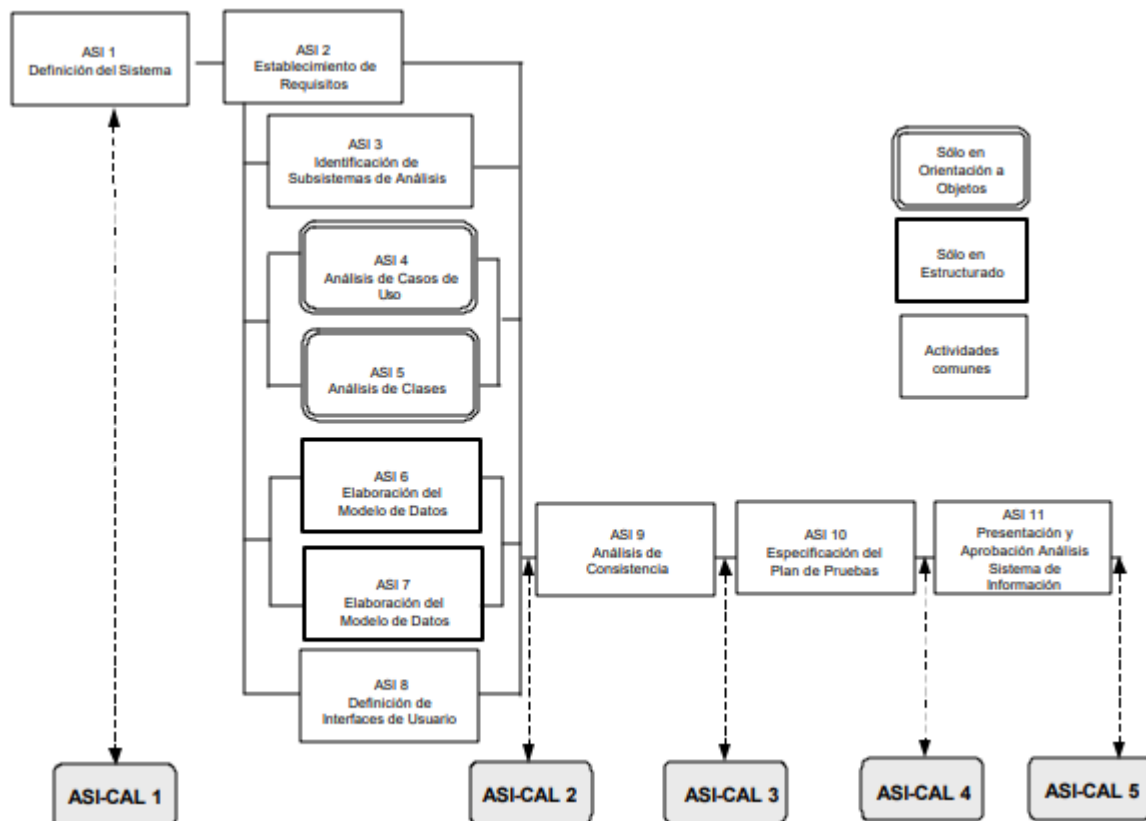


Ilustración 3. Controles en el análisis del sistema de información. METRICA v3

6.3.3 DISEÑO DEL SISTEMA DE INFORMACIÓN

Las revisiones del diseño se centran en confirmar que los requisitos especificados en el proceso previo de “Análisis del Sistema de Información” se han traducido en una arquitectura conforme al entorno tecnológico seleccionado y prescrito como estándar.

Asimismo, se revisan los requisitos que deben cumplir los distintos niveles de pruebas (unitarias, de integración, del sistema, de implantación y aceptación) especificados en el plan de pruebas, de acuerdo a los criterios de revisión establecidos en el plan de aseguramiento de calidad. También se realiza una revisión de la identificación de los requisitos no funcionales relacionados con la documentación de usuario e implantación.

El siguiente esquema muestra la correspondencia entre las actividades del proceso DSI y las de la interfaz de Aseguramiento de la Calidad.

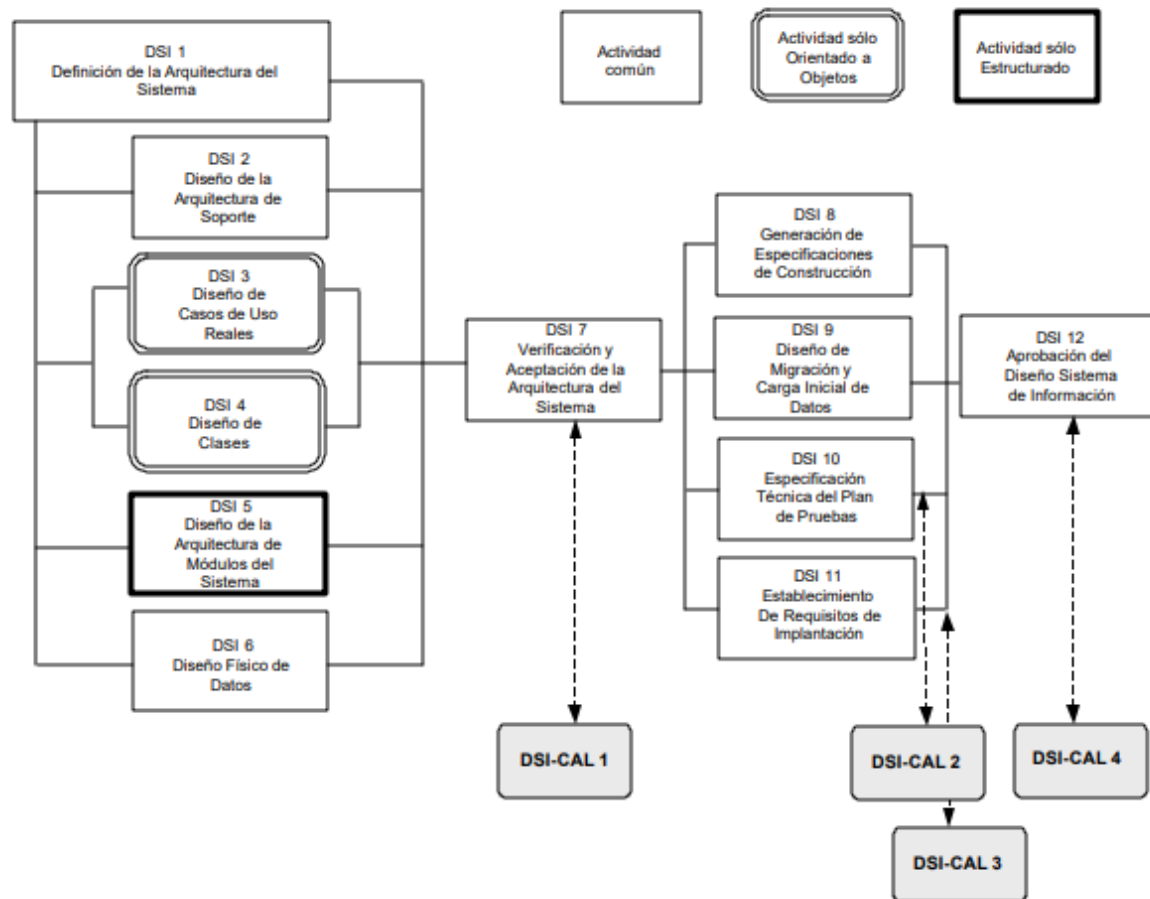


Ilustración 4. Controles en el diseño del sistema de información. METRICA v3

6.3.4 CONSTRUCCIÓN DEL SISTEMA DE INFORMACIÓN

En este proceso el grupo de aseguramiento de calidad revisa los estándares de nomenclatura y normativa aplicada en la generación del código de componentes, en la evaluación de los resultados de las pruebas, en los manuales de usuario y en el esquema de formación. Con respecto a las pruebas, se revisa que se han llevado a cabo las pruebas unitarias, de integración y del sistema según los criterios de selección de verificaciones y casos de prueba asociados que se habrán fijado en el plan de aseguramiento de calidad.

El siguiente esquema muestra la correspondencia entre las actividades del proceso CSI y las de la interfaz de Aseguramiento de la Calidad.

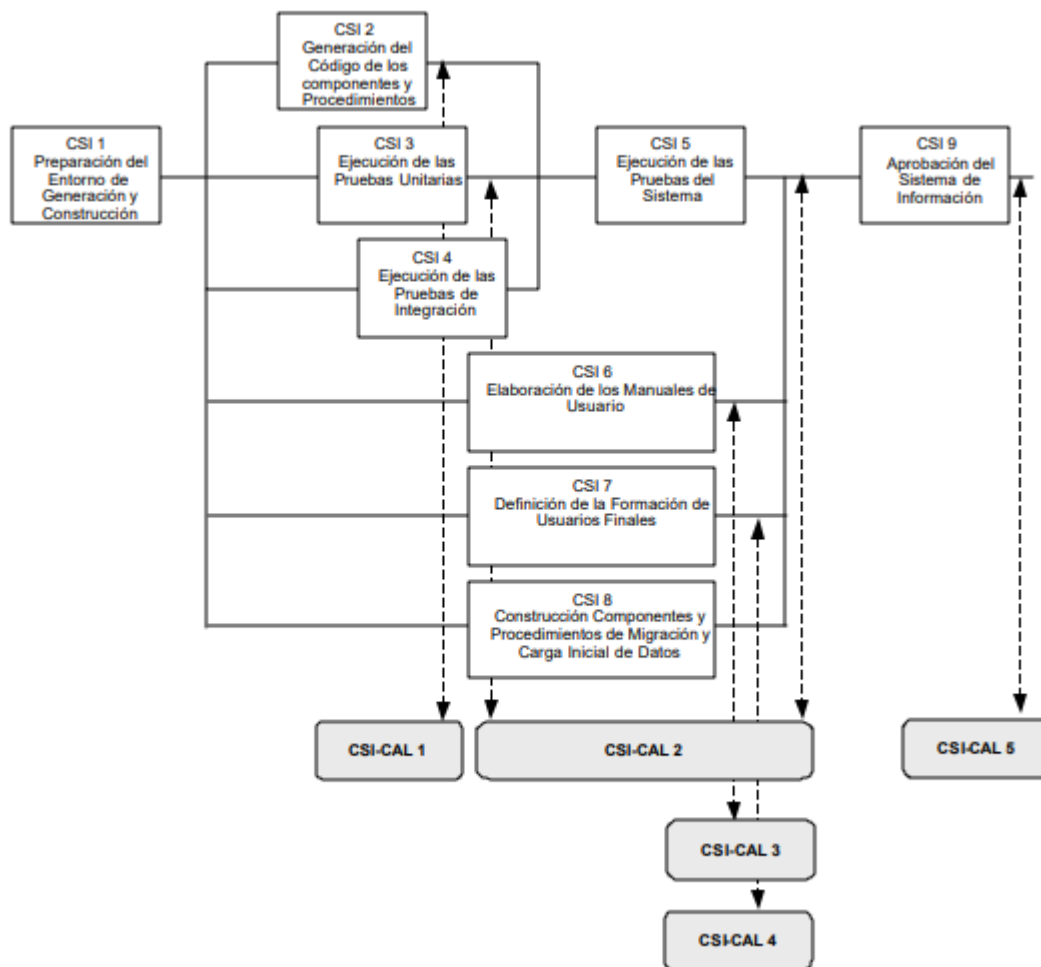


Ilustración 5. Controles en la construcción del sistema de información. METRICA v3

6.3.5 IMPLANTACIÓN Y ACEPTACIÓN DEL SISTEMA

El grupo de aseguramiento de calidad en este proceso se responsabiliza de revisar la existencia de un plan de implantación que se habrá elaborado conforme a la estrategia de implantación determinada en el proceso Estudio de Viabilidad del Sistema (EVS) y teniendo en cuenta los requisitos de implantación establecidos en el proceso Diseño del Sistema de Información (DSI).

También deben comprobar que se han realizado las pruebas de implantación y de aceptación según el plan de pruebas establecido en MÉTRICA Versión 3 y la normativa acordada en el plan de aseguramiento de calidad. Revisan la totalidad de las verificaciones y casos de prueba de implantación y aceptación que se hayan especificado para el sistema y las incidencias producidas, con el fin de determinar si puede verse afectada alguna propiedad de calidad.

En cualquier caso, se registra la aprobación de las pruebas de implantación y de aceptación por parte de operación y del usuario respectivamente.

En cuanto al mantenimiento, el grupo de aseguramiento de calidad debe asegurar que se le entrega el producto software al responsable de mantenimiento, con las propiedades adecuadas para que pueda asumir el servicio de mantenimiento, una vez que el sistema se encuentre en producción.

El siguiente esquema muestra la correspondencia entre las actividades del proceso IAS y las de la interfaz de Aseguramiento de la Calidad.

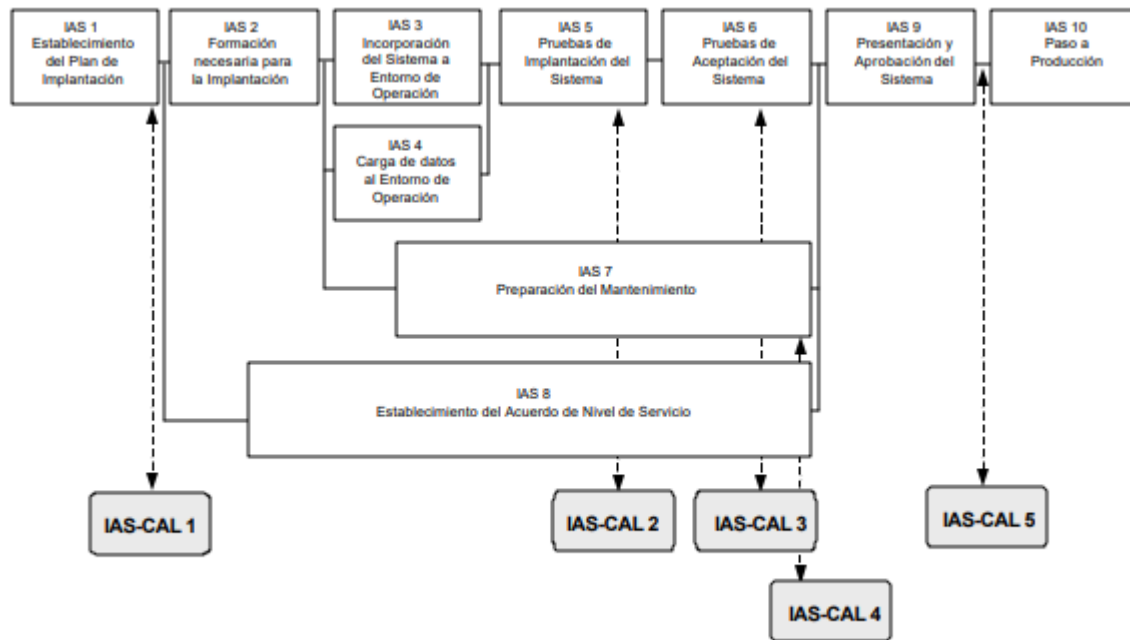


Ilustración 6. Controles en la implantación del sistema de información. METRICA v3

6.3.6 MANTENIMIENTO DEL SISTEMA DE INFORMACIÓN

En el proceso Implantación y Aceptación del Sistema se habrá determinado la necesidad de llevar a cabo un seguimiento y control de la calidad en los sistemas de información, una vez se encuentren en el entorno de producción.

El grupo de aseguramiento de calidad intervendrá durante el mantenimiento, efectuando revisiones de seguimiento periódicas, más o menos frecuentes según los casos, que sirvan para constatar que el mantenimiento establecido para el sistema de información se realiza de forma correcta.

En algún caso, según las implicaciones del cambio, puede ser necesario revisar puntualmente:

- El contenido del plan de pruebas de regresión.
- La ejecución de las pruebas de regresión según la normativa acordada en el plan de aseguramiento de calidad.
- Las verificaciones y casos de prueba que se hayan incluido en el plan de pruebas para los cambios producidos por una petición.
- Las incidencias detectadas con el fin de determinar si puede verse afectada alguna propiedad de calidad.

En caso de revisar la ejecución de las pruebas de regresión, se registrará la aprobación de las pruebas por el responsable de mantenimiento.

En el siguiente gráfico se aprecian las actividades de Aseguramiento de la Calidad durante el Mantenimiento del Sistema de Información.

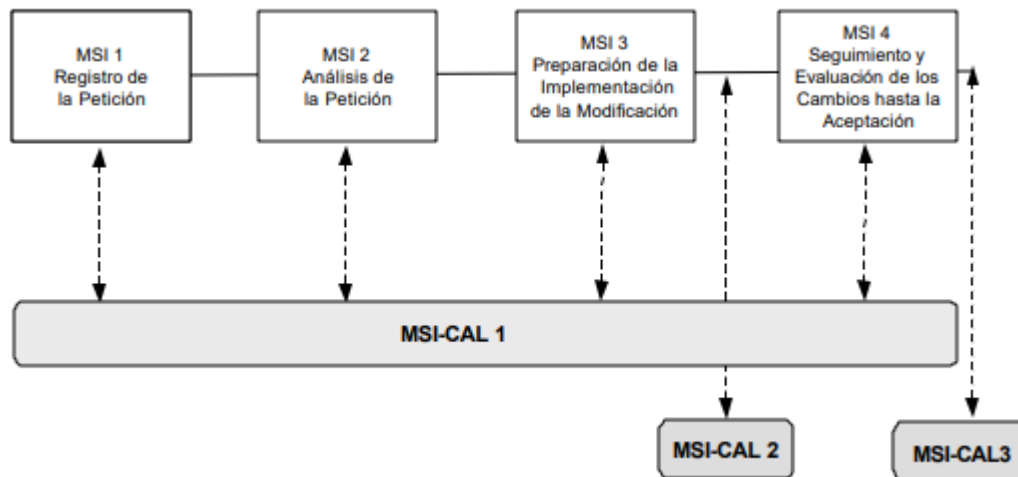


Ilustración 7. Controles en el mantenimiento del sistema de información. METRICA v3

7. HERRAMIENTAS DE CALIDAD DEL SW

En este apartado vamos a hacer una presentación de diferentes herramientas clasificadas por el tipo de prueba o métrica que miden y valoran.

Primero analizamos las herramientas de seguimiento de defectos de código:

- *SonarQube* es una herramienta de código abierto para la inspección continua de código. Recopila y analiza el código fuente y proporciona informes sobre la calidad del código de los proyectos. Aborda muchas partes de control de calidad del código como errores de estilo, errores potenciales, defectos de código, duplicación de código, falta de cobertura de prueba y exceso de complejidad. A su vez permite la conexión con otras herramientas para poder definir un catálogo de estas propios completamente adaptable.
- *Kiuwan* es una herramienta de revisión de código online basada en análisis estático. Facilita indicadores de alto nivel, sobre riesgo, calidad y defectos. Permite detectar errores de código críticos y vulnerabilidades de seguridad en su software.
- *PMD*, *FindBugs*, *CheckStile*, son herramientas de análisis de código estático especializadas en detectar errores en lenguaje Java.

También tendremos herramientas para la ejecución de determinados tipos de pruebas, tales como:

- Pruebas funcionales
 - *Selenium* es una popular herramienta de prueba de software de código abierto compatible con la mayoría de los navegadores y sistemas operativos. Permite simular el comportamiento de un usuario humano que prueba cualquier aplicación web a través de ese navegador. Ofrece compatibilidad con múltiples lenguajes de programación como Java, JavaScript, Python, C# y más.
 - *Katalon Studio*, construido sobre *Selenium WebDriver*, se considera su continuidad, con mejoras de uso y de alcance. Proporciona plantillas y acciones comunes para las pruebas basadas en palabras clave, se puede integrar con múltiples herramientas para la integración continua y tiene un panel rico y lleno de análisis. Es completamente gratuito, admite pruebas web y móviles, se ejecuta tanto en Windows como en Mac OS, tiene una comunidad activa y una interfaz de usuario amigable para principiantes.
 - Además de las anteriores, hay una infinidad de herramientas de prueba comerciales, tales como:
 - *TestComplete* es una plataforma funcional de pruebas automatizadas desarrollada por *SmartBear Software*. *TestComplete* permite a los evaluadores crear pruebas automatizadas para aplicaciones de Microsoft Windows, Web, Android (sistema operativo) e iOS. operaciones y se utiliza para la reproducción automatizada y el registro de errores.
 - *Testsigma* es una plataforma completa de automatización de pruebas basada en la nube que permite a los equipos ágiles simplificar y acelerar el control de calidad para aplicaciones web, aplicaciones móviles y API.
- Pruebas de API. Entre ellas destacan:
 - *SoapUI* es una herramienta de código abierto utilizada para pruebas de servicios web. Admite todos los protocolos y tecnologías estándar para probar todo tipo de API. Su interfaz es muy sencilla lo que facilita el uso tanto a usuarios como a técnicos.
 - *Postman* es un entorno de desarrollo de API libre que ayuda a los desarrolladores a crear, probar, documentar, monitorear y publicar documentación para sus API. Puede

realizar varios tipos de solicitudes HTTP (GET, POST, PUT, PATCH), guardar entornos para su uso posterior y convertir la API en código para varios idiomas como *JavaScript* y *Python*.

- Pruebas de rendimiento:
 - *JMeter* es una herramienta del proyecto Apache, libre y gratuita. Propone primero diseñar un “testplan”, fichero de extensión “.jmx”, con el diseño de la prueba definida agregando componentes de manera visual. Después permite ejecutar la prueba diseñada, variando los parámetros de número de usuarios o clientes, forma de arranque, etc. y generar registro de los tiempos de respuesta, errores, detectados, y mostrar estas estadísticas. Permite usar muchos protocolos y conexiones a diferentes servicios externos.
 - *LoadRunner* es una herramienta propietaria que permite detectar problemas de rendimiento de los componentes clave de las aplicaciones web. Es muy útil para detectar cuellos de botella en una fase previa a la implementación o despliegue de la aplicación, así como para identificar brechas de rendimiento antes de implementar o actualizar un nuevo sistema.
 - *LoadNinja* es una herramienta de pago para pruebas de rendimiento y carga basada en la nube para aplicaciones y servicios web. Permite verificar si sus servidores web soportan una carga masiva y si los servidores son robustos y escalables.
- Pruebas de seguridad
 - *W3af (Web Application Attack and Audit Framework)* es una aplicación web de código abierto que se utiliza para el análisis de seguridad. Esta herramienta proporciona un escáner de vulnerabilidades y herramientas de explotación para aplicaciones web. El escáner permite identificar más de 200 vulnerabilidades, incluidas las secuencias de comandos entre sitios, la inyección de SQL o el comando del sistema operativo.
 - *OpenVAS (vulnerability scanner)*. *OpenVAS*, desarrollado e impulsado por la empresa *Greenbone*, es un escáner de vulnerabilidades libre. Sus capacidades incluyen pruebas autenticadas y no autenticadas, varios protocolos industriales y de Internet de alto y bajo nivel, ajuste del rendimiento para escaneos a gran escala y un potente lenguaje de programación interno para implementar cualquier tipo de prueba de vulnerabilidad. El escáner obtiene las pruebas para detectar vulnerabilidades de una fuente o semilla (*feed*) que tiene un largo historial y actualizaciones diarias. Además, la empresa ofrece otras soluciones complementarias y de pago.
 - *Arachni (web application scanner)* *Arachni* es un *framework* Ruby que ayuda a los probadores de penetración y administradores a evaluar la seguridad de las aplicaciones web. Es inteligente, se entrena monitoreando y aprendiendo del comportamiento de la aplicación web durante el proceso de escaneo y es capaz de realizar metaanálisis utilizando una serie de factores para evaluar correctamente la confiabilidad de los resultados e identificar (o evitar) de manera inteligente los falsos positivos. Tiene en cuenta la naturaleza dinámica de las aplicaciones web, puede detectar cambios causados mientras viaja a través de las rutas de la complejidad ciclomática de una aplicación web y es capaz de ajustarse en consecuencia. De esta manera, los vectores de ataque / entrada que de otro modo serían indetectables por los no humanos se pueden manejar sin problemas. Además, debido a su entorno de navegador integrado, también puede auditar e inspeccionar el código del lado del cliente, así como admitir aplicaciones web altamente complicadas que hacen un uso intensivo de tecnologías como JavaScript, HTML5, manipulación DOM y AJAX.
 - *Acunetix Web Vulnerability Scanner* es una herramienta automatizada de pago para pruebas de seguridad de aplicaciones web que audita sus aplicaciones web

comprobando vulnerabilidades como inyección SQL, secuencias de comandos entre sitios y otras vulnerabilidades explotables.

- *Yasuo* (vulnerability scanner for web applications) Yasuo es un script en lenguaje “ruby” que busca aplicaciones web vulnerables de terceros. Yasuo está diseñado para escanear rápidamente la red en busca de tales aplicaciones vulnerables.
- Bases de datos de errores:
 - *Exploit-db*: En este repositorio hay más de 10,000 vulnerabilidades explotables remotamente que existen en toneladas de aplicaciones web / *front-ends* y podrían permitir que un atacante comprometa completamente el servidor *back-end*. Estas vulnerabilidades van desde mejoras de rendimiento o de relación coste /rendimiento, hasta cargas de archivos maliciosos, inyección SQL a RFI / LFI, etc.
 - Existen otras muchas fuentes de paquetes para explotar vulnerabilidades: packetstormsecurity.com, rapid7.com, cvedetails.com, securityfocus.com
- Herramientas de monitorización externa (todas como SW o servicios de pago), que publican dinámicamente el estado del comportamiento de una aplicación: Además de la monitorización, proponen apartados, subsistemas y comunicaciones de rendimiento mejorable o propuestas de mejora de configuración y de despliegue:
 - *DynaTrace*
 - *Amazon CloudWatch*
 - *Cisco Intersight Workload Optimizer*
 - *Crayon Cloud-iQ*

8. CONCLUSIONES

A lo largo de este tema se ha analizado la calidad del SW, primero presentado el concepto como especialidad de la calidad de cualquier otro producto, pero haciendo especial énfasis a la naturaleza virtual o no material del mismo, que dependen de un entorno de ejecución y que al ser un producto obtenido por fases requiere la calidad de la documentación previa a su implementación. De manera accesoria se analiza la calidad del proceso de desarrollo, no del producto final.

Después se ha realizado una evolución de diferentes modelos o estándares de calidad de SW, teniendo en cuenta su evolución, lo que nos focaliza los aspectos más iniciales de los que actualmente resultan más significativos y hacen foco los últimos modelos propuestos.

Se ha dedicado un espacio a definiciones relacionadas con la calidad de SW o con problemas concretos de código concretos analizado. En cuanto a las métricas de calidad definidas previamente, se han abordado su clasificación y descripción desde dos puntos de vista. Primero del producto intermedio o final analizado, con una aproximación de abajo a arriba, separando entre la documentación asociada y previa, el propio código fuente, el diseño o de arquitectura del sistema. También se ha analizado otras métricas de aspectos específicos de la experiencia de ejecución del SW (complejidad, rendimiento, mantenibilidad, accesibilidad, etc.). Finalmente se han enunciado métricas no del producto SW sino del proceso de desarrollo.

Posteriormente se ha abordado la calidad desde el punto de vista de principios de calidad, en las fases de ideación y de diseño del SW para dar cumplimiento a unos requisitos previos. Así hemos analizado dos de los principios más valorados y referidos: SOLID y patrones de diseño. Desde el punto de vista de los errores detectados, se han analizado las listas de errores más comunes en el SW. También se han introducido la necesidad de realización de pruebas de calidad del SW.

Se ha incluido igualmente un apartado sobre herramientas para la gestión automatizada de la calidad de SW, deparadas por el tipo de pruebas a realizar, con pruebas de seguridad, de rendimiento, funcionales, etc.

Finalmente, la calidad debe implantarse en un equipo de desarrollo. Se ha realizado una doble aproximación: de la constitución de un equipo específico de control de calidad en una organización TIC genérica, y del cumplimiento de Métrica 3 y de su conexión con el control de calidad en cada proyecto a abordar, separando en este segundo caso, las acciones a realizar en cada fase de la metodología.

9. RESUMEN ESQUEMÁTICO

- Concepto de calidad del SW. Se trata de un producto complejo de alto coste que debemos mantener durante su ciclo de vida completo.
- Importancia y especificidad de la calidad del SW, respecto de otros productos físicos y del coste de mantenimiento de estos principios.
- Conceptos previos sobre SW para medir su calidad. Naturaleza no física, importancia del mantenimiento posterior.
- Modelos de calidad del SW. Evolución de estándares reconocidos internacionalmente. ISO/IEC 12207, ISO/IEC 29110, ISO/IEC 25000 y ISO/IEC 9000.
- Métricas de calidad. Clasificación desde diferentes puntos de vista sobre la calidad de productos SW. Se han dividido en relativas a documentación, del propio código fuente, de los documentos, diseños usados y sobre la arquitectura definida. Además, se citan: métricas de preferencia, de complejidad, de desempeño, de exactitud, de rendimiento, de eficiencia, de configuración, de usabilidad, de accesibilidad, de defectos de seguridad y de mantenimiento.
- Evaluación de la calidad: principios de calidad, principios “solid”, patrones de diseño (creación, comportamiento y estructurales), errores comunes del SW y pruebas de SW.
- Implantación de una función la calidad: definición de normas y estándares y constitución del equipo de calidad. Aquí, hay que considerar las ventajas e inconvenientes de la creación de un equipo específico externo para la realización de pruebas o la introducción de los controles, métodos y herramientas específicos en el equipo de cada proyecto.
- Instalación de calidad por proyecto, según METRICA v3. La calidad se define como un “interface” previsto en métrica, que supone la revisión de los productos intermedios en cada fase prevista en la metodología. En cuanto a la calidad del SW, se recoge tanto en la fase de construcción como en la de implantación.
- Herramientas para pruebas de calidad por cada tipo de prueba a ejecutar. Para detectar defectos: SonarQube, Kiuwan, PMD, FindBugs, CheckStyle. Para pruebas funcionales: Selenium y Katalon Studio. Para pruebas de rendimiento: JMeter, LoadRunner y LoadNinja. Para pruebas de seguridad se pueden usar: W3af (Web Application Attack and Audit Framework), OpenVAS (vulnerability scanner), Arachni, Acunetix Web Vulnerability Scanner y Yasuo. Bases de datos de errores se debe citar exploit-db. Por último, pueden ser útiles herramientas de monitorización externa.
- Conclusiones que refuerzan las ventajas de implantar una función de calidad específica en cualquier organización dedicada a la producción de SW.

10. GLOSARIO

- SW. Software.
- TIC: Tecnologías de la información y las comunicaciones
- Calidad: “grado en que un conjunto de características inherentes cumple con unos requisitos” o “Conjunto de propiedades y características de un producto o servicio, que le confieren aptitud para satisfacer una serie de necesidades explícitas o implícitas” (ISO 8402)”.
- Calidad del software se puede definir también como: “Conformidad a los requisitos funcionales y prestaciones establecidas, a las normas de desarrollo explícitamente documentadas y a las características implícitas que se esperan de todo software desarrollado profesionalmente”
- Métrica: Métrica es una medida (cuantitativa) del grado, cantidad o tamaño que un producto o un proceso ofrecen. En el ámbito de desarrollo de SW y control de calidad una métrica es cualquier medida o conjunto de medidas que permiten caracterizar un software o sistema de información.
- Términos referidos en el apartado 4.1 que actúa como glosario introductorio de los siguientes apartados
- Patrones de diseño de creación, de estructura y de comportamiento.
- Principios de calidad SOLID
- EVS: ESTUDIO DE VIABILIDAD DEL SISTEMA
- ASI: ANÁLISIS DEL SISTEMA DE INFORMACIÓN
- DSI: DISEÑO DEL SISTEMA DE INFORMACIÓN
- CSI: CONSTRUCCIÓN DEL SISTEMA DE INFORMACIÓN
- IAS: IMPLANTACIÓN Y ACEPTACIÓN DEL SISTEMA
- MSI: MANTENIMIENTO DEL SISTEMA DE INFORMACIÓN
- PMD: *Programming Mistake Detector*). Herramienta para detectar problemas de calidad del SW.
- Yasuo (vulnerability scanner for web applications.
- W3af (Web Application Attack and Audit Framework)

11. TEST

11.1 PREGUNTAS DE TEST

- 1) El nombre del Proyecto SPICE son las siglas de:
 - a) *Software Process Investment and Capability Determination.*
 - b) *Software Process Improvement and Capability Determination.*
 - c) *Software Process Improvement and Complexity Determination.*
 - d) *Software Process Investment and complexity Determination.*
- 2) Según R Pressman, podemos definir la calidad del SW como
 - a) Conformidad a los requisitos funcionales y técnicos.
 - b) Conformidad a los requisitos funcionales, a prescripciones y a las características implícitas que se esperan de todo software independientemente del método de desarrollo.
 - c) Conformidad a los requisitos funcionales y prestaciones establecidas, a las normas de desarrollo explícitamente documentadas y a las características implícitas que se esperan de todo software desarrollado profesionalmente.
 - d) Conformidad a las características implícitas que se esperan de todo software desarrollado profesionalmente.
- 3) Cuál NO es un tipo de complejidad como métrica de calidad:
 - a) Ciclomática.
 - b) Cognitiva.
 - c) Computacional.
 - d) Aparente.
- 4) Al principio de que la configuración debe estar separada del código es un principio de calidad.
 - a) Del código.
 - b) De la documentación.
 - c) De la arquitectura.
 - d) Del diseño.
- 5) El patrón de diseño “*Singleton*” consiste en:
 - a) Crear objetos a partir de patrones.
 - b) Definir una única instancia de esa clase para toda la instalación.
 - c) Usar factorías estáticas.
 - d) Definir métodos de configuración para la creación de objetos en el sistema.
- 6) El patrón estructura “proxy” consiste en:
 - a) Proporcionar una fachada de interfaz no desarrollado, pendiente de implementación por clases hijas.
 - b) Definir una instancia a través de una factoría abstracta o no implementada.
 - c) Proporcionar un sustituto o representante de otro objeto para controlar el acceso a éste.
 - d) Proporcionar un adaptador entre un objeto y su cliente.
- 7) El número de “clicks” de ratón para alcanzar una funcionalidad se considera una métrica de:
 - a) Usabilidad.
 - b) Accesibilidad.
 - c) Rendimiento.
 - d) Definir métodos de configuración para la creación de objetos en el sistema

- 8) Las pruebas, por el grado de conocimiento del entorno y del sistema a probar se clasifican en varios apartados. ¿Cuál de los siguientes NO es un tipo de prueba?
- a) blanca.
 - b) negra.
 - c) gris o semitransparente.
 - d) opaca.
- 9) En relación con los principios de buen diseño, señale la respuesta correcta:
- a) Se deben declara objetos a través de clases y no de interfaces.
 - b) Se debe aumentar la cohesión y reducir el acoplamiento.
 - c) Se deben definir clases con varias responsabilidades.
 - d) No se deben definir interfaces con pocos métodos.
- 10) Las métricas de exactitud miden
- a) la precisión y validez de cada clase o paquete.
 - b) El tiempo de respuesta.
 - c) El número de errores por cada mil líneas de código.
 - d) Los métodos abstractos de una clase
- 11) La norma de calidad específica para implantar en empresas medias o pequeñas es:
- a) ISO/IEC 12207.
 - b) ISO/IEC 15504.
 - c) ISO/IEC 29110.
 - d) ISO 20.000.

11.2 SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
1	b
2	c
3	d
4	a
5	b
6	c
7	a
8	d
9	b
10	a
11	c

12. BIBLIOGRAFÍA BÁSICA

12.1 BIBLIOGRAFÍA BÁSICA

1. Normas ISO de calidad.

https://es.wikipedia.org/wiki/ISO/IEC_12207

https://es.wikipedia.org/wiki/ISO/IEC_15504

https://es.wikipedia.org/wiki/ISO/IEC_29110

2. Roger Pressman. Ingeniería del SW.

3. Metodología Métrica V3:

https://administracionelectronica.gob.es/pae_Home/pae_Documentacion/pae_Metodolog/pae_Metrica_v3.html

4. Repositorios de errores de SW comunes:

<https://owasp.org/API-Security/editions/2023/en/0x03-introduction/>

5. El libro de los 4: patrones de diseño:

[https://es.wikipedia.org/wiki/Patrones_de_dise%C3%B1o_\(libro\)](https://es.wikipedia.org/wiki/Patrones_de_dise%C3%B1o_(libro))

https://es.wikipedia.org/wiki/Patr%C3%B3n_de_dise%C3%B1o

6. Principios SOLID <https://es.wikipedia.org/wiki/SOLID>

7. Herramientas de calidad del SW:

<https://sentr.io/blog/herramientas-de-calidad-de-software/>

12.2 BIBLIOGRAFÍA PARA AMPLIAR EL TEMA

1. Ingeniería del SW: https://es.wikipedia.org/wiki/Ingenier%C3%ADa_de_software

2. Principios de diseño:

https://web.archive.org/web/20150906155800/http://www.objectmentor.com/resources/articles/Principles_and_Patterns.pdf

3. Calidad del SW: https://es.wikipedia.org/wiki/Calidad_de_software

4. Listados de errores de SW comunes: https://es.wikipedia.org/wiki/Anexo:Errores_de_software